TARAS SHEVCHENKO NATIONAL UNIVERSITY OF KYIV

FACULTY OF MECHANICS AND MATHEMATICS
DEPARTMENT OF ALGEBRA AND COMPUTER MATHEMATICS

Educational level: Master's

MASTER'S THESIS
submitted in fulfillment of the requirements for the degree of Master of Mathematics

on the topic

"Combinatorial Optimization of Robustness in Asynchronous Threshold Signatures: A Case Study of the FROST Extension"

by the 2nd-year Master's student
field of knowledge 11 Mathematics and Statistics
specialty 111 Mathematics
educational program Mathematics
Stanislav Oleksandrovych Karashchuk

Supervisor
Doctor of Physical and Mathematical Sciences, Professor
of the Department of Algebra and Computer Mathematics
Andrii Stepanovych Oliinyk

Admitted for defense before the Examination Board

Minutes No. _____ of the meeting of the

Department of Algebra and Computer Mathematics

dated _____ (date)

Head of the Department of Algebra and Computer Mathematics

_____ (signature)

_____ (name)

Kyiv, 2026

CONTENTS

INTRODUCTION

Relevance of the topic. Digital signatures underlie authentication in banking systems, blockchain networks, and electronic document workflows. Without them it is impossible to confirm the integrity or origin of data.

In the classical signature scheme, the secret key is held by a single user. If this key is compromised, security is lost entirely — this is a single point of failure. Threshold signature schemes (TSS) [1] remove this problem: the secret is distributed among n participants, and signing requires the participation of any t of them (t — a predetermined threshold).

Among threshold Schnorr signature schemes, the FROST protocol (Flexible Round-Optimized Schnorr Threshold Signatures) [2] has become widely adopted, reducing signing to two rounds and allowing unbounded parallelism of sessions. However, FROST sacrifices robustness for efficiency: if the coordinator selects a group of t signers and at least one of them fails to respond or sends incorrect data, the entire signing session fails. In an asynchronous network, where it is impossible to distinguish a malicious participant from network delay, this leads to a state of infinite waiting, making the system vulnerable to ”denial of service” attacks.

Formally, the robustness of a threshold signature scheme is defined as the guarantee that, given at least t honest signers, the protocol will complete successfully and produce a valid signature, regardless of the behavior of the remaining participants. To achieve this property in an asynchronous setting, the ROAST protocol (Robust Asynchronous Schnorr Threshold Signatures) has been proposed [3], which is a wrapper over FROST. ROAST maintains a set of ”active” signers and automatically forms new signing sessions, guaranteeing completion in no more than $n - t + 1$ sessions.

At the same time, open questions remain regarding the formalization of the quorum selection problem, obtaining precise combinatorial bounds on the number of signing rounds, and constructing probabilistic models that account for participant heterogeneity and coordinator strategies. These questions define the direction of this thesis.

State of research on the problem. Threshold signature schemes originate from the work of Shamir [4] on secret sharing and Feldman [5] on verifiable secret sharing. Pedersen [6] constructed the first distributed key generation (DKG) scheme without a trusted dealer. Later, Gennaro et al. [7] discovered a vulnerability in it: a malicious participant can manipulate the distribution of the shared secret by filing complaints after inspecting other participants' shares.

Based on the Schnorr signature [8], Komlo and Goldberg [2] constructed the two-round threshold protocol FROST. Crites, Komlo, and Maller [9] proved its security in the discrete logarithm model. The security of two-round multisignatures was separately analyzed by Drijvers et al. [10]. Ruffing et al. [3] added to FROST the ROAST extension, which guarantees completion of signing even in the presence of malicious participants. The result of Fischer, Lynch, and Paterson [11] on the impossibility of consensus with even one faulty process outlines the theoretical limit for such protocols.

At the same time, the existing literature has not addressed the formalization of the quorum selection problem in the language of hypergraphs, has not proven the optimality of the $n-t+1$-session bound for ROAST, and has not constructed probabilistic models with heterogeneous participant response intensities and adaptive coordinator strategies.

The object of research is asynchronous threshold Schnorr signature schemes, and **the subject of research** is the combinatorial and probabilistic characteristics of the robustness of the FROST and ROAST protocols in the presence of malicious participants.

The aim of the research is to develop a mathematical apparatus for analyzing and optimizing the robustness of asynchronous threshold signatures, using the FROST extension as a case study.

Research objectives:

1. Describe the algorithm of the FROST protocol for (t, n)-threshold signing on the Edwards25519 curve;
2. Replace the classical Pedersen DKG with the modular ChillDKG protocol [12] and prove statements regarding its security;
3. Describe the ROAST extension over the FROST protocol to ensure robustness in asynchronous networks;
4. Formalize the quorum selection problem as a problem on t-uniform hypergraphs;
5. Obtain a combinatorial estimate of the upper bound on the number of rounds for successful signing in the presence of f malicious participants and prove its optimality;
6. Construct a probabilistic model of the system using absorbing Markov chains to analyze coordinator strategies;
7. Generalize the model to the case of heterogeneous response intensities and investigate the dependence of signing time on the fraction of malicious participants;

Scientific novelty of the results obtained.

1. The quorum selection problem for ROAST is formalized in the language of t-uniform hypergraphs.
2. It is proven that the upper bound of $n - t + 1$ sessions is optimal, by constructing an adaptive adversary strategy that forces exactly f failed sessions.
3. Based on absorbing Markov chains, a formula for the expected number of sessions and a formula for the expected completion time are obtained.
4. The model is generalized to heterogeneous response intensities of participants; a reputation system is introduced.
5. For ChillDKG, five statements regarding security are proven: resistance to threshold escalation, correctness and completeness of adversary identification, public verifiability of proofs of malicious behavior, the impossibility of identifiable abort in 1 round, and the optimality of 3 logical rounds.

Practical significance of the results obtained. The combinatorial estimates and the probabilistic model make it possible to make an informed choice of parameters (t, n) for threshold cryptographic systems in blockchain infrastructure (Bitcoin Taproot addresses), distributed key storage systems, and multisignature protocols. The open-source implementation in Go [13] is suitable for integration into applied systems.

Structure and scope of the thesis. The thesis consists of an introduction, five chapters, conclusions, and a list of references.

Chapter 1 contains preliminaries: digital signatures, elliptic-curve cryptography on Edwards25519, the Shamir and Feldman secret-sharing schemes, threshold signature schemes, and Schnorr signatures.

Chapter 2 is devoted to the FROST protocol: the commitment-binding mechanism, the signing phase with two polynomials, and the aggregation of partial signatures.

In **Chapter 3**, the ChillDKG protocol is considered as a replacement for Pedersen DKG: the vulnerabilities of the earlier scheme are analyzed, and statements regarding security and round optimality are proven.

Chapter 4 describes the ROAST extension, which makes threshold signing robust in asynchronous networks.

In **Chapter 5**, the quorum selection problem is formalized via t-uniform hypergraphs, the optimality of the $n - t + 1$-session bound is proven, and a probabilistic model based on absorbing Markov chains is constructed, generalized to heterogeneous response intensities.

CHAPTER 1. PRELIMINARIES

In this chapter we describe the mathematical apparatus underlying threshold signature schemes and the FROST protocol: the structure of digital signatures, elliptic-curve cryptography, secret-sharing schemes, and Schnorr signatures.

Definition 1.1 (Digital signature scheme)**.** A digital signature scheme is a triple of polynomial-time algorithms (KeyGen, Sign, Verify)

- KeyGen$(1^\lambda) \to (sk, pk)$ - generation of a key pair: secret key sk and public key pk, where λ is the security parameter;
- Sign$(sk, m) \to \sigma$ - computation of a signature σ on a message m using the secret key sk;
- Verify$(pk, m, \sigma) \to \{0, 1\}$ - signature verification, returning 1 if the signature σ is valid for message m with respect to the public key pk, and 0 otherwise.

A scheme is **correct** if for any pair $(sk, pk) \leftarrow$ KeyGen(1^λ) and any message m we have Verify$(pk, m,$ Sign$(sk, m)) = 1$.

Definition 1.2 (Edwards curve)**.** Let $\mathbb{F}_q$ be a finite field with q elements, where q is prime, $q \equiv 1 \pmod 4$, and $d \in \mathbb{F}_q$ is a non-square element. An Edwards curve is the set

$$E = \left\{(x, y) \in \mathbb{F}_q \times \mathbb{F}_q : -x^2 + y^2 = 1 + dx^2y^2\right\}$$

with the addition operation

$$(x_1, y_1) + (x_2, y_2) = \left(\frac{x_1y_2 + x_2y_1}{1 + dx_1x_2y_1y_2}, \frac{y_1y_2 + x_1x_2}{1 - dx_1x_2y_1y_2}\right)$$

and identity element $\mathcal{O} = (0, 1)$. The set E forms an abelian group under this operation.

This thesis uses the Edwards25519 curve [15] with parameters:

- $q = 2^{255} - 19$;
- $d = -121665/121666 \in \mathbb{F}_q$;
- subgroup order $l = 2^{252} + 27742317777372353535851937790883648493$;
- base point $B = (x, 4/5) \in E$, where $x > 0$.

Curve25519 is the Montgomery curve $v^2 = u^3 + 486662u^2 + u$, birationally equivalent to the Edwards curve [16, 17] via the substitutions

$$x = \sqrt{486664} \cdot \frac{u}{v}, \quad y = \frac{u - 1}{u + 1}.$$

The security of elliptic-curve cryptographic protocols is based on the computational hardness of the discrete logarithm problem.

Definition 1.3 (Elliptic curve discrete logarithm problem)**.** Let $(\mathbb{G}, +)$ be a cyclic group of order l with generator G. Given G and $P = [x]G$, where $x \in \mathbb{Z}_l$, the discrete logarithm problem (ECDLP) is to find x.

For the Edwards25519 curve with a 256-bit scalar, the best known algorithms require on the order of 2^{128} operations [18], which makes recovering the secret key x from the public key $P = [x]G$ computationally infeasible.

EdDSA is a generalized digital signature scheme on Edwards curves [15, 19]. For the Edwards25519 curve, the scheme parameters are: $b = 256$, $H =$ SHA-512.

Keys. The secret key is a b-bit string k. The hash $H(k) = (h_0, h_1, \ldots, h_{2b-1})$ defines the scalar

$$a = 2^{b-2} + \sum_{3 \le i \le b-3} 2^i h_i,$$

which determines the public key $A = [a]B$. The lower bits $h_b, \ldots, h_{2b-1}$ are used to generate the nonce during signing.

Signing. A signature on message M with key k is formed as follows:

1. $r = H(h_b, \ldots, h_{2b-1}, M) \bmod l$;
2. $R = [r]B$;
3. $S = (r + H(R, A, M) \cdot a) \bmod l$;
4. the signature is the pair (R, S).

Verification. For a signature (R, S) on message M with public key A:

1. compute $c = H(R, A, M) \bmod l$;
2. check the equality $[S]B \stackrel{?}{=} R + [c]A$.

Correctness follows from the identity $S = r + c \cdot a \pmod{l}$, whence $[S]B = [r]B + [c \cdot a]B = R + [c]A$.

In the FROST-Ed25519 protocol, a group of n participants $P_1, \ldots, P_n$ holds Shamir shares s_i of a secret scalar s, where the shares are represented as elements of $\mathbb{Z}_q$, $q = l$. Any subset of at least t distinct shares allows s to be reconstructed. Participants obtain their shares through a distributed key generation (DKG) protocol, together with the group key $A = [s] \cdot G$ and public shares $\{A_i = [s_i] \cdot G\}$.

A FROST-Ed25519 signature on message M is a pair (R, S), where

$$R = [r] \cdot G, \quad c = \mathrm{H}\big(R \,\|\, A \,\|\, M\big) \bmod q, \quad S = (r + c \cdot s) \bmod q.$$

The verification algorithm takes $(A, M, (R, S))$ and checks

$$R \stackrel{?}{=} [-c] \cdot A + [S] \cdot G,$$

Definition 1.4 (Shamir (t, n)-threshold secret-sharing scheme [4])**.** Let $\mathbb{F}_q$ be a finite field with q elements, q prime. For a secret $s \in \mathbb{F}_q$, the dealer chooses random coefficients $a_1, \ldots, a_{t-1} \stackrel{\$}{\leftarrow} \mathbb{F}_q$ and constructs the polynomial

$$f(x) = s + \sum_{i=1}^{t-1} a_i x^i \in \mathbb{F}_q[x],$$

where $f(0) = s$. The share of participant P_i is defined as $s_i = f(i)$ for $i \in \{1, \ldots, n\}$.

To reconstruct the secret, any subset $T \subseteq \{1, \ldots, n\}$, $|T| \ge t$, performs Lagrange interpolation:

$$s = f(0) = \sum_{i \in T} s_i \cdot \Lambda_{T,i}, \quad \text{where} \quad \Lambda_{T,i} = \prod_{j \in T \setminus \{i\}} \frac{j}{j - i}.$$

The coefficients $\Lambda_{T,i}$ are called **Lagrange coefficients**.

The Shamir scheme does not allow verification of the correctness of received shares. This shortcoming is removed by Feldman's verifiable secret sharing (Feldman VSS) [5].

Definition 1.5 (Feldman verifiable secret sharing)**.** In addition to distributing private shares $(i, f(i))$ to each participant P_i, the dealer publishes a vector of commitments

$$\vec{C} = \langle \phi_0, \phi_1, \ldots, \phi_{t-1} \rangle, \quad \text{where} \quad \phi_j = [a_j]G \quad (a_0 = s).$$

Participant P_i verifies their share s_i by checking

$$[s_i]G \stackrel{?}{=} \sum_{j=0}^{t-1} [i^j]\phi_j.$$

In practice, each participant must confirm that their $\vec{C}$ matches what other participants received. Commitments are usually published through a coordinator or compared in a separate round.

Definition 1.6 (Additive secret sharing)**.** Let α participants $P_1, \ldots, P_\alpha$ independently choose values $s_i \in \mathbb{F}_q$. The shared secret is defined as

$$s = \sum_{i=1}^{\alpha} s_i.$$

Additive sharing requires no interaction. Cramer, Damgård, and Ishai [20] showed that additive shares $s = \sum_i s_i$ can be locally converted into polynomial form via Lagrange coefficients: $s_i = \lambda_i \cdot \hat{s}_i$.

FROST uses this technique during signing to non-interactively generate a nonce, Shamir-shared among participants.

Definition 1.7 ((t, n)-threshold signature scheme)**.** A threshold signature scheme is a scheme in which the secret key s is distributed among n participants using a (t, n)-threshold secret-sharing scheme, and a single public key $Y = [s]G$ represents the group. Any subset T with $|T| \geq t$ participants can produce a signature that verifies under the key Y, while no coalition of $|T'| < t$ participants can reconstruct s or forge a signature.

Definition 1.8 (Quorum)**.** A quorum in a (t, n)-threshold signature scheme is an arbitrary subset $T \subseteq \{1, \ldots, n\}$ with $|T| = t$ participants, sufficient to produce a valid threshold signature. The total number of possible quorums equals $\binom{n}{t}$. The notion of quorum is used in this thesis to analyze the robustness of the ROAST protocol (Chapter 5), where the coordinator's task reduces to adaptively searching for a quorum consisting exclusively of honest participants, as well as for the combinatorial and probabilistic analysis (Chapter 6), where quorums are formalized as hyperedges of a t-uniform hypergraph.

The FROST protocol requires the resulting signature to conform to the standard Schnorr signature scheme [8]. Since threshold schemes guarantee that no group of fewer than t participants learns the secret key s, the generation of s and the distribution of shares $s_1, \ldots, s_n$ are performed via a distributed key generation (DKG) protocol.

Moreover, the nonce k must also be generated in a distributed manner — each participant makes a contribution, but no one knows k in full. For this, FROST relies on additive secret sharing.

Definition 1.9 (Distributed key generation (DKG))**.** A distributed key generation protocol is a protocol after whose completion each participant P_i obtains a secret share s_i and a shared public key $Y = [s]G$, where $s = f(0)$ for some polynomial f of degree $t - 1$, such that no participant knows s in full, and no group of fewer than t participants can reconstruct it.

Pedersen [6, 21] proposed a two-round DKG in which each participant plays the role of the dealer in Feldman's VSS [5]. Each participant P_i chooses a secret x_i, broadcasts the commitment $[x_i]G$, and sends the corresponding secret shares to the other participants. The result is a shared secret defined as $s = \sum_{i=1}^{n} x_i$.

Gennaro et al. [7] discovered a vulnerability in Pedersen's DKG [6]: a malicious participant can distort the distribution of the shared secret by filing complaints against an honest participant after inspecting their shares. To eliminate this

vulnerability, the authors add a commitment round that fixes the value of the secret before the contributions are revealed, increasing the number of rounds to three. Section 3.1 contains a more detailed analysis of the problems with Pedersen DKG and the attack found by auditors in 2024.

Signatures produced as a result of FROST's threshold operations are standard Schnorr signatures and are verified by the ordinary verification algorithm [9].

Definition 1.10 (Schnorr signature)**.** Let $(\mathbb{G}, p, g)$ be a group of prime order p with generator g, and let H be a cryptographic hash function $H\colon \{0,1\}^* \to \mathbb{Z}_p$. For a secret key $s \in \mathbb{Z}_p$ and public key $Y = g^s$, a signature on message m is generated as follows:

1. choose a random nonce $k \xleftarrow{\$} \mathbb{Z}_p$; $R = g^k$;
2. $c = H(R, Y, m)$;
3. $z = k + s \cdot c \in \mathbb{Z}_p$;
4. the signature is $\sigma = (R, z)$.

Verification of signature $\sigma = (R, z)$ on message m with public key Y:

1. $c = H(R, Y, m)$;
2. $R' = g^z \cdot Y^{-c}$;
3. the signature is valid if and only if $R = R'$.

Proposition 1.1. The correctness of the Schnorr signature follows from the identity

$$g^z \cdot Y^{-c} = g^{k+sc} \cdot g^{-sc} = g^k = R.$$

The Schnorr signature is a non-interactive proof of knowledge of the discrete logarithm Y (a Σ-protocol), made non-interactive via the Fiat-Shamir transform [22].

CHAPTER 2. THE FROST PROTOCOL

The FROST protocol [2] implements threshold Schnorr signatures over two rounds of interaction among signers. Below we examine the tradeoff between efficiency and robustness built into FROST, the role of the signature aggregator, the key generation and signing procedures, and the verification of partial signatures.

FROST is designed for the "optimistic" case: all participants are honest. If someone violates the protocol, the rest can determine who exactly, and restart the session without them. That is, FROST deliberately sacrifices robustness in order to reduce the number of rounds.

FROST introduces a semi-trusted signature aggregator (SA) role, which reduces communication overhead. The protocol also works without an SA — participants simply broadcast messages to one another.

The aggregator can be any participant, or even an external party, as long as it knows the public shares Y_i. The SA collects partial signatures, reports violators, and publishes the final signature. Even if the SA behaves dishonestly, security is not compromised: the SA has no privileged access to secrets.

Before signing, participants must run a DKG. Afterward, each P_i obtains a long-term share (i, s_i), and the group obtains a public key Y. The public share $Y_i = [s_i]G$ is needed to verify partial signatures; the group key Y allows external parties to verify signatures.

As in any protocol using Feldman VSS, participants must agree on commitments $\vec{C}_i$ for $1 \leq i \leq n$. Figure 1 shows the classical KeyGen based on Pedersen DKG; in our implementation it is replaced by ChillDKG (Chapter 4), which removes the limitations of the classical approach (see Chapter 3.1).

Gennaro et al. [7, 23] showed that Ped-DKG with restart ("Stop, Kill, and Rewind") is vulnerable to adversarial manipulation. In practice, however, once a violation is detected, its causes should be investigated immediately. DKG performance is not critical — it is a one-time operation performed when generating long-term keys.

Let us consider the FROST signing protocol. This operation is based on well-known methods from the literature [5], applying additive secret sharing and share conversion to non-interactively generate a nonce value for each signature.

KeyGen (Key Generation)

Round 1. Each P_i chooses $(a_{i0}, \ldots, a_{i(t-1)}) \leftarrow \mathbb{Z}_q$, constructs $f_i(x) = \sum_{j=0}^{t-1} a_{ij}x^j$, and computes a proof of knowledge of a_{i0}: $k \leftarrow \mathbb{Z}_q$, $R_i = [k]G$, $c_i = H(i, \Phi, [a_{i0}]G, R_i)$, $\mu_i = k + a_{i0} \cdot c_i$, where Φ is a context string. VSS commitment: $\vec{C}_i = \langle C_{i0}, \ldots, C_{i(t-1)} \rangle$, $C_{ij} = [a_{ij}]G$. P_i sends $\vec{C}_i$ and $\sigma_i = (R_i, \mu_i)$ to the others. Each P_i, upon receiving $\vec{C}_l, \sigma_l$ from P_l, checks: $R_l \stackrel{?}{=} [\mu_l]G - [c_l]C_{l0}$, $c_l = H(l, \Phi, C_{l0}, R_l)$. On failure, the protocol aborts.

Round 2. Each P_i sends P_l the share $(l, f_i(l))$, keeping only $(i, f_i(i))$. Check: $[f_l(i)]G \stackrel{?}{=} \sum_{k=0}^{t-1}[i^k]C_{lk}$. Private share: $s_i = \sum_{l=1}^{n} f_l(i)$, public: $Y_i = [s_i]G$. Group key: $Y = \sum_{j=1}^{n} C_{j0}$. Public share of another participant: $Y_i = \sum_{j=1}^{n}\sum_{k=0}^{t-1}[i^k]C_{jk}$.

Figure 1: FROST distributed key generation protocol (KeyGen)

Figure 1 — the classical Pedersen DKG [6] with proofs of possession. In our implementation it is replaced by ChillDKG (Chapter 4).

The protocol consists of two rounds. First, each participant chooses a secret x_i, broadcasts a commitment and a proof of possession $\sigma_i = (R_i, \mu_i)$; then sends secret shares to the others. The proof of possession protects against rogue-key attacks; on violation the protocol halts. On output, everyone shares a common public key $Y = \sum_{j=1}^{n} C_{j0}$, while each participant holds only their own share $s_i = \sum_{l=1}^{n} f_l(i)$, and fewer than t participants are unable to reconstruct s.

2.1 Preprocessing Stage

Preprocess$(\pi) \rightarrow \big(i, \langle (D_{ij}, E_{ij}) \rangle_{j=1}^{\pi}\big)$

Each P_i, $i \in \{1, \ldots, n\}$, generates π nonce pairs prior to signing. For $1 \leq j \leq \pi$: (a) choose $(d_{ij}, e_{ij}) \xleftarrow{\$} \mathbb{Z}_q^* \times \mathbb{Z}_q^*$; (b) compute $(D_{ij}, E_{ij}) = ([d_{ij}]G, [e_{ij}]G)$; (c) store $((d_{ij}, D_{ij}), (e_{ij}, E_{ij}))$. Publish (i, L_i), where $L_i = \langle (D_{ij}, E_{ij}) \rangle_{j=1}^{\pi}$.

Figure 2: FROST preprocessing protocol

Figure 2 — the preprocessing stage. Participants generate and publish π commitments at once, allowing π signatures to be performed before the next preprocessing round. If caching is undesirable, a two-round signing protocol can be used instead, in which commitments are published in the first round.

Specifically, each P_i generates a list of one-time private nonce pairs and the corresponding public commitment shares

$$\Big\langle \Big((d_{ij}, D_{ij} = [d_{ij}]G),\ (e_{ij}, E_{ij} = [e_{ij}]G) \Big) \Big\rangle_{j=1}^{\pi},$$

where j is a counter identifying the next nonce/commitment-share pair available for use when signing. Each P_i then publishes (i, L_i), where $L_i = \big\langle (D_{ij}, E_{ij}) \big\rangle_{j=1}^{\pi}$ is its list of commitment shares. The set of tuples (i, L_i) is stored by any entity that may act as the signature aggregator during signing.

2.2 The Signing Protocol

FROST's signing operations use the binding technique [10] to prevent known forgery attacks without limiting parallelism. The attack described by Drijvers et al. requires the attacker either to observe the victim's commitment values before choosing its own commitment, or to adaptively choose the message to be signed, allowing the adversary to manipulate the resulting challenge c. To prevent this attack, FROST "binds" each participant's response to the specific message, as well as to the set of participants and their commitments used for the given signing operation, via the hash $\rho_i = H_1(i, m, B)$. Thus, combining responses for different messages, or for different participant/commitment pairs, results in an invalid signature.

At the start of the signing protocol (Figure 3), the signature aggregator SA selects α participants, where $t \leq \alpha \leq n$, to take part in signing. Let S denote the set of these α participants. SA selects the next available commitment (D_i, E_i) for each $i \in S$ and forms an ordered list $B = \langle (i, D_i, E_i) \rangle_{i \in S}$, then sends each P_i the tuple (m, B).

The resulting secret nonce is defined as $k = \sum_{i \in S} k_i$, where each $k_i = d_i + e_i \cdot \rho_i$, and (d_i, e_i) correspond to the values $(D_i = [d_i]G, E_i = [e_i]G)$ published during preprocessing. Since the k_i are additive shares of k, they are Shamir shares of k with Lagrange coefficients λ_i for the i-th participant over the set S.

Each participant's response z_i to the challenge is computed using the one-time nonces (d_i, e_i) and the long-term secret share s_i, converted to additive form:

$$z_i = d_i + (e_i \cdot \rho_i) + \lambda_i \cdot s_i \cdot c.$$

SA verifies the correctness of each z_i against the corresponding commitment share (D_i, E_i) and public key share Y_i. If every participant produced a correct z_i, the group response is defined as $z = \sum_{i \in S} z_i$, and the group signature on message m is $\sigma = (R, z)$. This signature is verified using the standard Schnorr signature verification operation with the public key Y.

Since every nonce and commitment share generated during preprocessing must be used at most once, participants must delete these values after using them in a signing operation. Accidental reuse of (d_{ij}, e_{ij}) can lead to disclosure of the participant's long-term secret s_i.

Sign$(m) \to (m, \sigma)$

Let SA be the signature aggregator, S the set of α participants ($t \leq \alpha \leq n$), Y the group public key, $B = \langle (i, D_i, E_i) \rangle_{i \in S}$ the ordered list of commitments, H_1, H_2 hash functions with outputs in $\mathbb{Z}_q^*$.

1. SA fetches the next available commitment for each $P_i \in S$ from L_i and forms B.
2. For each $i \in S$, SA sends P_i the tuple (m, B).
3. Each P_i checks m and that $D_l, E_l \neq \mathcal{O}$ for every commitment in B.
4. Each P_i computes: $\rho_l = H_1(l, m, B)$ for $l \in S$, $R = \sum_{l \in S}(D_l + [\rho_l]E_l)$, $c = H_2(R, Y, m)$.
5. Each P_i computes the response: $z_i = d_i + e_i \cdot \rho_i + \lambda_i \cdot s_i \cdot c$, where λ_i is the Lagrange coefficient for i with respect to S.
6. P_i deletes $((d_i, D_i), (e_i, E_i))$ and returns z_i to the aggregator SA.
7. SA performs:
   - **7.a.** Compute $\rho_i = H_1(i, m, B)$, $R_i = D_i + [\rho_i]E_i$, $R = \sum_{i \in S} R_i$, $c = H_2(R, Y, m)$.
   - **7.b.** Check: $[z_i]G \stackrel{?}{=} R_i + [c \cdot \lambda_i]Y_i$ for each $i \in S$. If not, identify the violator.
   - **7.c.** Compute $z = \sum_{i \in S} z_i$.
   - **7.d.** Publish $\sigma = (R, z)$ together with m.

Figure 3: Single-round FROST signing protocol

The construction of FROST relies on two polynomials. $F_1(x)$ defines the sharing of the private key s (where $Y = [s]G$), while $F_2(x)$ defines the sharing of the nonce k:

$$k = \sum_{i \in S} \big(d_i + e_i \cdot \rho_i\big),$$

where ρ_i is derived from the public data (m, B). During key generation, the first polynomial is constructed as

$$F_1(x) = \sum_{j=1}^{n} f_j(x),$$

so that the secret key shares take the form $s_i = F_1(i)$, and the secret key equals $s = F_1(0)$.

During signing, the α chosen participants ($t \leq \alpha \leq n$) construct, from nonce pairs (d_i, e_i), a polynomial $F_2(x)$ of degree $\alpha - 1$ interpolating $(i,\ d_i + e_i \cdot H_1(i, m, B))$:

$$F_2(0) = \sum_{i \in S} \lambda_i \big(d_i + e_i \cdot \rho_i\big),$$

where λ_i are the Lagrange coefficients for i over S. Let $F_3(x) = F_2(x) + c \cdot F_1(x)$ with $c = H_2(R, Y, m)$. Then for $i \in S$

$$z_i = d_i + \big(e_i \cdot \rho_i\big) + \lambda_i \cdot s_i \cdot c = \lambda_i \Big(F_2(i) + c \cdot F_1(i)\Big) = \lambda_i F_3(i).$$

Thus the group response $z = \sum_{i \in S} z_i$ is the Lagrange interpolation of the value $F_3(0)$, i.e.,

$$F_3(0) = \left(\sum_{i \in S} \big(d_i + e_i \cdot \rho_i\big)\right) + c \cdot s.$$

Since $R = \big[\sum_{i \in S}(d_i + e_i \cdot \rho_i)\big]G$ and $c = H_2(R, Y, m)$, the pair (R, z) is a valid Schnorr signature on message m.

Despite its efficiency, FROST has a robustness problem in asynchronous networks. The aggregator fixes a quorum of t signers at the start of a session. If even one of them is malicious or simply slow, the sum $z = \sum_{i \in S} z_i$ cannot be

computed — the session must be restarted with a different set.

The reason is the FLP theorem [11]: in an asynchronous setting, a slow node cannot be distinguished from a failed node. A single malicious participant can block signing indefinitely.

The FROST signing and verification algorithm is conveniently presented in the following semi-interactive form, shown in Figure 4. We use this exact scheme in (Chapter 5) as a primitive for the ROAST protocol.

Setup $\Sigma =$ FROST2

PreRound(PK):
$d_i, e_i \xleftarrow{\$} \mathbb{Z}_q;\ (D_i, E_i) \leftarrow ([d_i]G, [e_i]G)$
$\text{state}_i \leftarrow (d_i, e_i);\ \rho_i \leftarrow (D_i, E_i)$
return (state_i, ρ_i)

PreAgg$(PK, \{\rho_i\}_{i\in T})$:
$B \leftarrow \langle (i, D_i, E_i)\rangle_{i\in T};$ **return** $\rho \leftarrow B$

SignAgg$(PK, \rho, \{\sigma_i\}_{i\in T}, m)$:
$B \leftarrow \rho;$ **for** $i \in T$: $\rho_i \leftarrow H_1(i, m, B)$
$R \leftarrow \sum_{i\in T}(D_i + [\rho_i]E_i);\ s \leftarrow \sum_{i\in T}\sigma_i$
return $\sigma \leftarrow (R, s)$

Lagrange(T, i):
return $\lambda_i \leftarrow \prod_{j\in T\setminus\{i\}} \frac{j}{j-i}$

SignRound$(sk_i, PK, T, \text{state}_i, \rho, m)$:
$(d_i, e_i) \leftarrow \text{state}_i;\ B \leftarrow \rho$
for $l \in T$: $\rho_l \leftarrow H_1(l, m, B)$
$R \leftarrow \sum_{l\in T}(D_l + [\rho_l]E_l);\ c \leftarrow H_2(Y, m, R)$
$\lambda_i \leftarrow$ Lagrange(T, i)
$\sigma_i \leftarrow d_i + \rho_i \cdot e_i + c \cdot \lambda_i \cdot s_i;$ **return** σ_i

ShareVal$(PK, T, i, \rho, \rho_i, \sigma_i, m)$:
$(D_i, E_i) \leftarrow \rho_i;\ B \leftarrow \rho;\ \rho_i \leftarrow H_1(i, m, B)$
$R_i \leftarrow D_i + [\rho_i]E_i$
$R \leftarrow \sum_{l\in T}(D_l + [H_1(l, m, B)]E_l);\ c \leftarrow H_2(Y, m, R)$
$\lambda_i \leftarrow$ Lagrange$(T, i);$ **return** $[\sigma_i]G \stackrel{?}{=} R_i + [c\lambda_i]Y_i$

Verify(PK, m, σ):
$(R, s) \leftarrow \sigma;\ c \leftarrow H_2(Y, m, R);$ **return** $[s]G \stackrel{?}{=} R + [c]Y$

Figure 4: FROST2 signing and verification algorithms [3, 2]

The ChillDKG protocol [12] is proposed as a standalone alternative to the classical Pedersen DKG [6] for use together with FROST. Unlike Pedersen DKG, ChillDKG does not depend on an external secure broadcast channel and does not require an external consensus mechanism. Shares are encrypted via ECDH, the CertEq sub-protocol guarantees agreement of results among honest participants, and recovery data allows a lost share to be reconstructed without re-running the DKG. The security of the combination of SimplPedPop (the base primitive of ChillDKG) and FROST is proven in [24].

3.1 Limitations of the Classical Pedersen DKG

The classical Pedersen protocol [6] (Section 2.10) has several limitations that complicate its use in asynchronous settings. First of all, the protocol transmits partial secrets in the clear: in the second round each participant sends the share $f_i(j)$ to participant P_j without encryption, and intercepting even a single channel compromises the corresponding share.

Furthermore, after the protocol completes, honest participants have no cryptographic guarantee that everyone received identical commitments $\vec{C}_i$ (the so-called lack of equality checking). A coordinator or a malicious participant can send different sets of commitments to different participants, so that part of the group obtains one group key Y while the rest obtain another. To illustrate, consider a threshold wallet with a 2-of-3 scheme in which two participants are honest and the third is malicious. Suppose the malicious participant sends invalid shares to the first honest participant but valid ones to the second. Then the first honest participant cannot complete the DKG, while the second considers the protocol successful and begins accepting funds to the resulting threshold key. These funds are irrecoverably lost, since the second participant's single share is insufficient to produce a signature [12].

In addition, if a share fails VSS verification, an honest participant knows only that a failure occurred, but cannot unambiguously determine who is at fault — the dealer or the coordinator. Loss of a secret share s_i is also irrecoverable: the only way to recover it is to re-run the entire DKG protocol.

Finally, in February 2024, Dalgren (Trail of Bits) [25] disclosed a denial-of-service vulnerability affecting the Pedersen DKG phase in implementations of the FROST, GG20, and GG18 threshold schemes. The attack exploits the absence of a polynomial-degree check. In the Pedersen protocol, each participant P_i is supposed to generate a polynomial $p_i(x)$ of degree $t-1$ and publish a commitment vector $(A_{i,0}, A_{i,1}, \ldots, A_{i,t-1})$ of length t. If a malicious participant instead uses a polynomial of degree $T > t-1$, the group polynomial $p(x) = \sum_i p_i(x)$ will have degree T, and reconstructing the secret will require $T+1$ shares instead of t. If $T+1 > n$, the shared key becomes unusable: no coalition can produce a signature, and in key-resharing systems, funds tied to the shared key are irrecoverably lost. The research identified 10 vulnerable implementations, including the reference FROST implementation (Chelsea Komlo), ZCash Foundation FROST, and Penumbra FROST. The fix is trivial: it suffices to check the length of the commitment vector $|\vec{C}_i| = t$ for each participant. In ChillDKG this check is included at the SimplPedPop level, ensuring resistance to this attack by construction.

Because of these limitations, the classical Pedersen DKG is not suitable for threshold systems using ROAST.

3.2 Description of the ChillDKG Protocol

ChillDKG is built from three layers: SimplPedPop (basic VSS), EncPedPop (share encryption), and CertEq (finalization). Participants $P_1, \ldots, P_n$ communicate only through a relaying coordinator $\mathcal{C}$, which aggregates and forwards messages. The coordinator only handles encrypted shares and therefore has no access to secrets. Each P_i has a long-term key pair (hsk_i, hpk_i) with $hpk_i = [hsk_i]G$; the parameters params $= (\{hpk_i\}_{i=1}^n, t)$ are agreed upon in advance.

The SimplPedPop layer. The base layer implements the Pedersen scheme [24], augmented with a proof-of-possession mechanism. For each P_i, $i \in \{1, \ldots, n\}$, the initialization procedure includes: choosing a random polynomial $f_i(x) =$

$\sum_{j=0}^{t-1} a_{ij}x^j \in \mathbb{Z}_q[x]$; computing commitments $\vec{C}_i = \langle [a_{i0}]G, \ldots, [a_{i(t-1)}]G \rangle$; forming a signature $\pi_i = \text{Sign}_{a_{i0}}(i)$ as a PoP; generating shares $s_{ij} = f_i(j)$ for all $j \in \{1, \ldots, n\}$.

The coordinator collects $(\vec{C}_i, \pi_i, \{s_{ij}\})$ from all participants and computes the aggregated commitments $\hat{C}_k = \sum_{i=1}^{n} [a_{ik}]G$ for $k = 1, \ldots, t-1$ and the group key $Y = \sum_{i=1}^{n} [a_{i0}]G$.

Each P_j verifies the PoP via SchnorrVerify$([a_{i0}]G, i, \pi_i)$, computes the aggregated share $s_j = \sum_{i=1}^{n} s_{ij}$, and checks:

$$[s_j]G \stackrel{?}{=} \sum_{k=0}^{t-1} [j^k] \cdot \hat{C}_k, \quad \hat{C}_0 = Y.$$

The EncPedPop layer. The EncPedPop layer adds confidentiality of shares using ephemeral ECDH. Each P_i generates a pair $(r_i, R_i = [r_i]G)$. The shared secret for the pair (P_i, P_j) is:

$$\text{pad}_{ij} = H\big(\text{“encpedpop”} \,\|\, [r_i]hpk_j \,\|\, R_i \,\|\, hpk_j \,\|\, \text{ctx}\big).$$

The encrypted share is: $\hat{s}_{ij} = s_{ij} + \text{pad}_{ij} \pmod q$. The coordinator computes $\hat{s}_j = \sum_{i=1}^{n} \hat{s}_{ij}$ for each P_j; the homomorphism of addition prevents recovery of individual shares. Participant P_j decrypts:

$$s_j = \hat{s}_j - \sum_{i=1}^{n} \text{pad}_{ij},$$

where $\text{pad}_{ij} = H\big(\text{“encpedpop”} \,\|\, [hsk_j]R_i \,\|\, R_i \,\|\, hpk_j \,\|\, \text{ctx}\big)$ is computed from hsk_j and the public R_i.

The CertEq layer and finalization. The CertEq mechanism guarantees output integrity and conditional agreement properties [12].

Proposition 3.1 (Integrity)**.** If an honest participant P_i successfully completes CertEq with input x, then every other honest participant P_j who also completes CertEq has an input equal to x.

Proposition 3.2 (Conditional agreement)**.** If an honest participant P_i successfully completes CertEq and obtains a certificate cert, then any other honest participant P_j, upon receiving cert, can also successfully complete CertEq.

CertEq does not guarantee simultaneous completion, i.e., cert is a *deferred proof* of success, so a participant who was offline can later obtain cert and independently verify the correctness of the session.

After verifying the shares, each P_j computes the transcript eq_input $= (t \,\|\, \hat{C}_0 \,\|\, \ldots \,\|\, \hat{C}_{t-1})$ and signs it: $\sigma_j = \text{Sign}_{hsk_j}(\text{eq\_input} \,\|\, j)$. The coordinator forms a certificate cert $= (\sigma_1, \ldots, \sigma_n)$ and distributes it to all participants. If all n signatures are valid, each participant is convinced of the identity of the group key Y and the shares $\{Y_i\}$; an invalid signature σ_j unambiguously identifies P_j as malicious.

Separately, the protocol generates recovery data recovery $=$ (eq_input, cert). If share s_i is lost, participant P_i, having hsk_i and recovery, verifies the certificate, decrypts the share via ECDH, and verifies it against the VSS commitment. The complete protocol is shown in Figure 5.

ChillDKG$(hsk_i, \text{params}, n, t)$

Round 1 ($P_i \to \mathcal{C}$): each P_i generates $f_i \in \mathbb{Z}_q[x]$, deg $f_i = t-1$; computes $\vec{C}_i$, π_i; generates (r_i, R_i); encrypts shares $\hat{s}_{ij} = s_{ij} + \text{pad}_{ij}$; sends $\mathcal{C}$: $(\vec{C}_i, \pi_i, R_i, \{\hat{s}_{ij}\}_{j=1}^n)$.

Aggregation ($\mathcal{C}$): $\hat{C}_k = \sum_i [a_{ik}]G$, $\hat{s}_j = \sum_i \hat{s}_{ij}$; sends P_j: $(\{[a_{i0}]G\}, \{\hat{C}_k\}, \{\pi_i\}, \{R_i\}, \hat{s}_j)$.

Round 2 ($P_j \to \mathcal{C}$): each P_j verifies the PoP, decrypts s_j, checks $[s_j]G \stackrel{?}{=} \sum_k [j^k]\hat{C}_k$; computes eq_input; sends $\sigma_j = \text{Sign}_{hsk_j}(\text{eq\_input} \| j)$.

Finalization: $\mathcal{C}$ forms cert $= (\sigma_1, \ldots, \sigma_n)$; each P_j verifies all n signatures. Output: $s_j, Y, \{Y_i\}$, (eq_input, cert).

Figure 5: Complete ChillDKG protocol

Consider an adversary who controls up to $t-1$ participants and attempts to secretly raise the effective threshold of the group polynomial, so that reconstructing the secret would require more than t shares.

Theorem 3.1 (Resistance to threshold escalation)**.** Let $f_0, f_1, \ldots, f_{n-1}$ be the individual polynomials of n participants, where each f_j has degree at most $t-1$. Define the group polynomial $F(x) = \sum_{j=0}^{n-1} f_j(x)$. If the protocol enforces $|\vec{C}_j| = t$ for each participant j, then $\deg(F) \le t-1$, and any subset of t or more secret shares $\{F(i)\}$ suffices to reconstruct $F(0)$. No adversary controlling fewer than t participants can cause $\deg(F) > t-1$.

Proof.

Each participant P_j publishes a VSS commitment $\vec{C}_j$ of length $|\vec{C}_j| = t$. This commitment encodes a polynomial $f_j(x) = \sum_{k=0}^{t-1} a_{j,k} x^k$ of degree at most $t-1$. By the binding property of the discrete-logarithm commitment, P_j is computationally bound to a polynomial of degree at most $t-1$. Committing to a polynomial of degree $d \ge t$ would require a vector of length $d+1 > t$, which is rejected by the check $|\vec{C}_j| = t$.

$$F(x) = \sum_{j=0}^{n-1} f_j(x) = \sum_{k=0}^{t-1} \left(\sum_{j=0}^{n-1} a_{j,k} \right) x^k.$$

Since each f_j has degree at most $t-1$, the coefficient of x^k for $k \ge t$ is zero, hence $\deg(F) \le t-1$.

By the fundamental property of Shamir's scheme, a polynomial of degree at most $t-1$ over $\mathbb{Z}_q$ is uniquely determined by t evaluation points. The group secret $s = F(0)$ is recovered by Lagrange interpolation from any t shares:

$$F(0) = \sum_{i \in T} \Lambda_{T,i} \cdot F(i), \quad \Lambda_{T,i} = \prod_{\substack{j \in T \\ j \ne i}} \frac{j}{j-i}, \quad |T| = t.$$

An adversary controlling $|\mathcal{C}| \le t-1$ participants may choose their polynomials arbitrarily, but each of them satisfies $\deg(f_j) \le t-1$ due to the commitment-length check. Hence $\deg(F) \le t-1$ regardless of the adversary's strategy. □

Suppose a participant P_i discovers that their aggregated share does not match the aggregated public commitment, i.e., $[s_i]G \ne A_i$. In that case, an investigation procedure is triggered:

1. The coordinator discloses the partial public shares $\{A_{ji}\}_{j=0}^{n-1}$, where $A_{ji} = \sum_{k=0}^{t-1} [i^k] \cdot C_{j,k}$.
2. P_i checks $\sum_{j=0}^{n-1} A_{ji} = A_i$. If not, the coordinator is malicious.
3. For each j, P_i checks $[s_{ji}]G \stackrel{?}{=} A_{ji}$. If it fails for $j \ne i$, then P_j is malicious; if it fails for $j = i$, the coordinator is malicious.

Theorem 3.2 (Correctness of identification)**.** If participant P_j is honest (correctly computed $s_{ji} = f_j(i)$ and published a valid commitment $\vec{C}_j$), then Feldman's check $[s_{ji}]G = A_{ji}$ passes, and P_j is never identified as malicious.

Proof. By construction, $C_{j,k} = [a_{j,k}]G$ and $s_{ji} = f_j(i) = \sum_{k=0}^{t-1} a_{j,k} \cdot i^k$. Therefore:

$$[s_{ji}]G = \left[\sum_{k=0}^{t-1} a_{j,k} \cdot i^k\right] G = \sum_{k=0}^{t-1} [i^k] \cdot [a_{j,k}]G = \sum_{k=0}^{t-1} [i^k] \cdot C_{j,k} = A_{ji}.$$

The check at step 3 passes for P_j, hence P_j is not identified as malicious. □

Theorem 3.3 (Completeness of identification)**.** If the aggregated-share check fails ($[s_i]G \neq A_i$), the coordinator is honest, and all partial shares were correctly transmitted, then the investigation procedure identifies at least one malicious participant P_j with $j \neq i$.

Proof. Suppose the coordinator is honest. Then $\sum_j A_{ji} = A_i$ (step 2 passes) and $\sum_j s_{ji} = s_i$. Since $[s_i]G \neq A_i$:

$$\sum_{j=0}^{n-1} [s_{ji}]G \neq \sum_{j=0}^{n-1} A_{ji}.$$

By linearity of scalar multiplication, there exists j^* such that $[s_{j^*i}]G \neq A_{j^*i}$. Since the coordinator is honest, the share $s_{ii} = f_i(i)$ was computed locally and correctly by P_i, and $A_{ii} = \sum_{k=0}^{t-1} [i^k] C_{i,k}$ corresponds to a valid commitment $\vec{C}_i$, so $[s_{ii}]G = A_{ii}$ and $j^* \neq i$. Hence P_{j^*} is identified as malicious at step 3. □

Theorem 3.4 (Public verifiability)**.** The proof of malicious behavior $(i, j, s_{ji}, \vec{C}_j)$ is publicly verifiable: any third party, given the public commitment $\vec{C}_j$, can verify $[s_{ji}]G \neq \sum_{k=0}^{t-1} [i^k] \cdot C_{j,k}$ without needing any secret information.

Proof. The commitment $\vec{C}_j$ is public (broadcast during round 1). The index i is public. The partial share s_{ji} is disclosed by P_i during the investigation. The verification equation uses only public data. Disclosing s_{ji} does not compromise the aggregated share $s_i = \sum_j s_{ji}$, since the remaining shares $\{s_{j'i}\}_{j' \neq j}$ remain secret. □

Theorem 3.5 (Impossibility of identifiable abort in 1 round)**.** No 1-round protocol in the asynchronous model with a relaying coordinator can achieve identifiable abort (IA) if the coordinator may be malicious.

Proof. Suppose, for contradiction, that there exists a 1-round protocol Π_1 with IA. In such a protocol, each P_i sends a message $m_i^{(1)}$ to the coordinator, and the coordinator responds with a message $\hat{m}_i^{(1)}$ to each participant.

A malicious coordinator $\mathcal{C}^*$ performs a split-view attack: it partitions the honest participants into two groups $\mathcal{H}_1, \mathcal{H}_2$ and sends each group different messages. Since there is no broadcast channel, $P_i \in \mathcal{H}_1$ and $P_j \in \mathcal{H}_2$ cannot compare the messages they received within a single round.

From P_i's point of view, the execution of the protocol is indistinguishable between two scenarios: (a) the coordinator is honest, and some participant in $\mathcal{H}_2$ sent an invalid contribution; (b) the coordinator is malicious and forges the contributions of $\mathcal{H}_2$. In scenario (a), identifying the coordinator is a false accusation; in scenario (b), identifying a participant of $\mathcal{H}_2$ is a false accusation. Since P_i cannot distinguish between these scenarios, correct identification is impossible. □

Theorem 3.6 (Sufficiency of 3 logical rounds)**.** ChillDKG achieves identifiable abort in 3 logical rounds of communication (4 message flows: 2 from participant to coordinator, 2 from coordinator to participant) under the assumptions:

(A1) unforgeability of Schnorr signatures (EU-CMA);

(A2) binding of VSS commitments (hardness of the discrete logarithm in $\mathbb{G}$).

Proof. We analyze each failure mode:

At round 2, participant P_i verifies the Schnorr signature π_j for each P_j. By EU-CMA, an honest P_j always produces a valid signature; a verification failure unambiguously identifies P_j.

After decryption, P_i checks $[s_i]G \stackrel{?}{=} \sum_{k=0}^{t-1} [i^k] \cdot \hat{C}_k$. By the completeness-of-identification theorem, a failed check allows the malicious party to be identified.

Suppose $\mathcal{C}^*$ sends P_i the transcript x and P_j the transcript $x' \neq x$. At round 2, P_i signs x, P_j signs x'. To build a certificate under x, the coordinator needs a valid signature from P_j on x, but P_j signed $x' \neq x$. By EU-CMA, forging P_j's signature on x is computationally infeasible. Hence the certificate will contain an invalid signature, which is detected at round 3.

Round 1 collects commitments and encrypted shares. Round 2 allows each participant to verify the data and produce a signature as an unforgeable witness of their view of the transcript. Round 3 distributes these signatures, allowing discrepancies to be detected. □

Proposition 3.3 (Optimality of ChillDKG)**.** By the 1-round impossibility theorem, no protocol in the asynchronous model without a broadcast channel can achieve identifiable abort with fewer participant logical rounds than ChillDKG.

ChillDKG can identify a malicious participant, but does not guarantee robustness: a single dishonest or faulty participant disrupts the entire DKG [12].

First, ensuring robustness would require the coordinator to exclude non-responding participants, which already reduces the t-of-n scheme to $(t-1)$-of-$(n-1)$ from the outset. A malicious coordinator could abuse this by arbitrarily excluding honest participants. Even without a coordinator, achieving robustness under a dishonest majority ($t > n/2$) remains an open problem.

Second, from a practical standpoint, robustness is needed specifically at the *signing* stage, not at key generation. DKG is performed once, whereas signing is performed repeatedly. Therefore, robustness of signing is provided by the ROAST protocol (Chapter 5), while DKG remains non-robust: on failure, participants must investigate the cause and repeat the ceremony.

ROAST (Robust Asynchronous Schnorr Threshold Signatures) [3] — a wrapper that turns a semi-interactive threshold signature scheme Σ with identifiable abort into a robust asynchronous protocol. We use $\Sigma =$ FROST [2].

4.1 Model and Definition of Robustness

Signers $S_1, \ldots, S_n$ are connected to the coordinator $\mathcal{C}$ via reliable, authenticated point-to-point channels. The network is asynchronous: the only assumption is that messages between honest parties are eventually delivered [11].

Adversary model. The adversary controls $f \leq n-t$ signers (both during key generation and during signing), but not the coordinator. This model is a variant of the Byzantine fault model [26], in which malicious participants may behave arbitrarily.

Definition 4.1 ((n,t,f)-robustness)**.** Let $F \subseteq [n]$, $|F| = f$, be the set of indices of malicious signers. A threshold signing protocol is (n,t,f)-**robust** if for any message m and keys $(PK, (sk_1, \ldots, sk_n))$ obtained via the key generation protocol, in any execution of the signing protocol the coordinator $\mathcal{C}$ eventually terminates and outputs a signature σ such that Verify$(PK, \sigma, m) = 1$, provided that the adversary controls the message-delivery schedule but is required to deliver messages between honest parties [3].

Definition 4.2 (Robustness)**.** A threshold signing protocol is **robust** if for all $t \leq n$ and $f = n - t$ it is (n,t,f)-robust.

The value $f = n-t$ is optimal: no unforgeable threshold protocol can be robust for $f > n-t$, since $n-f \leq t-1$ signers are unable to produce a signature.

ROAST uses as a primitive a semi-interactive scheme Σ with algorithms:

- PreRound$(PK) \to (\rho_i, \text{state}_i)$ - generation of a pre-signature share;
- PreAgg$(PK, \{\rho_i\}_{i \in T}) \to \rho$ - aggregation of pre-signature shares;
- SignRound$(sk_i, PK, T, \text{state}_i, \rho, m) \to \sigma_i$ - computation of a partial signature;
- ShareVal$(PK, T, i, \rho, \rho_i, \sigma_i, m) \to \{0,1\}$ - verification of a partial signature;
- SignAgg$(PK, \rho, \{\sigma_i\}_{i \in T}, m) \to \sigma$ - aggregation of partial signatures into the final signature.

The property of **identifiable abort**[1] (IA-CMA) means: an honest signer's response always passes ShareVal, and a failed check indicates a malicious party.

While the original paper uses a variant called FROST3 [3] as Σ, in this thesis we use FROST2, described earlier in this thesis, whose specification is given in Figure 4 (Chapter 3). The main difference is that in FROST2, $\rho_i = H_1(i, m, B)$ is computed from the full list of commitments $B = \langle (i, D_i, E_i) \rangle_{i \in T}$.

4.2 The ROAST Algorithm

The coordinator $\mathcal{C}$ maintains a set R of *active* signers - those who have responded to all previous requests. As soon as $|R| = t$, the coordinator initiates a new signing session Σ with these participants. Along with the partial signature σ_i for the current session, each signer also sends a fresh pre-signature share ρ_i' for a future session, forming a pipeline of sessions.

[1] IA-CMA (Identifiable Abort under Chosen Message Attack) — if a session aborts, honest participants unambiguously identify at least one malicious party.

Coordinator $\mathcal{C}(PK, n, t, m)$

$R \leftarrow \emptyset$; $M \leftarrow \emptyset$; $P[\,] \leftarrow$ array(n); sidctr $\leftarrow 0$

$SID[\,] \leftarrow$ array(n); $T[\,], N[\,], S[\,] \leftarrow$ array$(n{-}t{+}1)$

upon receive (σ_i, ρ'_i) **from** $S_i, i \notin M$:

1. **if** $i \in R$ **then** MarkMalicious(i); **break**
2. **if** $SID[i] \neq \bot$ **then**:
   (a) $sid \leftarrow SID[i]$; $T_{sid} \leftarrow T[sid]$; $\rho \leftarrow N[sid]$; $\rho_i \leftarrow P[i]$
   (b) **if** $\neg$ ShareVal$(PK, T_{sid}, i, \rho, \rho_i, \sigma_i, m)$ **then** MarkMalicious(i); **break**
   (c) $S[sid] \leftarrow S[sid] \cup \{\sigma_i\}$
   (d) **if** $|S[sid]| = t$ **then** $\sigma \leftarrow$ SignAgg$(PK, \rho, S[sid], m)$; **return** σ
3. $P[i] \leftarrow \rho'_i$; $R \leftarrow R \cup \{i\}$
4. **if** $|R| = t$ **then**:
   (a) sidctr $\leftarrow$ sidctr $+ 1$
   (b) $\rho \leftarrow$ PreAgg$(PK, \{P[i]\}_{i \in R})$
   (c) **foreach** $i \in R$: **send** (ρ, R) **to** S_i; $SID[i] \leftarrow$ sidctr
   (d) T[sidctr] $\leftarrow R$; N[sidctr] $\leftarrow \rho$; $R \leftarrow \emptyset$

proc MarkMalicious(i): $M \leftarrow M \cup \{i\}$; **if** $|M| > n - t$ **then fail**

Figure 6: ROAST coordinator algorithm

Notation in Figure 6:

- R — the set of active signers (those not currently awaited in any session);
- M — the set of identified malicious participants;
- $P[i]$ — the latest pre-signature share ρ_i from signer S_i;
- sidctr — session counter;
- $SID[i]$ — the identifier of the session in which S_i is awaited ($\bot$ if not awaited);
- $T[sid], N[sid], S[sid]$ — the set of signers, the aggregated pre-signature, and the collected partial signatures of session sid, respectively;
- MarkMalicious(i) — adds S_i to M; if $|M| > n - t$, fewer than t honest signers remain and the protocol halts.

Signer $S_i(sk_i, PK, m)$

1. $(\rho_i, \text{state}_i) \leftarrow$ PreRound(PK)
2. **send** $(\bot, \rho_i)$ **to** $\mathcal{C}$

upon receive (ρ, R) **from** $\mathcal{C}$:

1. $\sigma_i \leftarrow$ SignRound$(sk_i, PK, R, \text{state}_i, \rho, m)$
2. $(\rho_i, \text{state}_i) \leftarrow$ PreRound(PK)
3. **send** (σ_i, ρ_i) **to** $\mathcal{C}$

Figure 7: ROAST signer algorithm

Notation in Figure 7: state_i — the signer's local secret state (the nonce pair (d_i, e_i), generated by PreRound); ρ_i — the pre-signature share (D_i, E_i) sent to the coordinator. During initialization, the signer sends $(\bot, \rho_i)$, where $\bot$ denotes the absence of a partial signature (the first session has not yet begun).

A malicious coordinator can launch several parallel sessions, so ROAST only provides *weak* unforgeability: the adversary may obtain several signatures on a single message. For Bitcoin, after SegWit, this is not a problem [3].

Theorem 4.1. Let Σ be a semi-interactive threshold signature scheme with the IA-CMA property[1]. Then ROAST(Σ) is robust, and the coordinator successfully terminates after initiating no more than $n - t + 1$ signing sessions of Σ [3].

Proof. We define auxiliary notions. A signer's response in a session is *valid* if it is not unsolicited (coordinator step 1) and passes the ShareVal check (step 2(b)). A session *completes* if the coordinator receives t valid responses. A signer is *awaited* in a session if it has not yet sent a valid response. A signer is *disruptive* if there exists a session in which it never sends a valid response.

By the IA-CMA property[1], honest signers always send valid responses, so they are never disruptive and are never added to the set M. Hence there are at most f disruptive signers.

The protocol maintains an invariant: each signer is awaited in at most one session at a time. This is ensured by the fact that awaited signers are not included in the set R (step 3) and therefore are not added to new sessions (step 4).

Suppose, for contradiction, that no session ever completes. Every incomplete session contains at least one disruptive signer among those awaited. Since each disruptive signer is awaited in at most one session, the number of simultaneously incomplete sessions never exceeds $f = n - t$.

At the same time, honest signers are not disruptive: their messages are eventually delivered, and they become active (join R). After initiating $k \leq n - t$ sessions, among the signers who are not currently awaited, at least $n - k \geq n - (n - t) = t$ participants remain, including all $n - f = t$ honest signers. As soon as t of them become active, the coordinator initiates the $(k + 1)$-th session. By the pigeonhole principle, when $k + 1 > f$ there are not enough disruptive signers to block all sessions, so at least one of them completes. The total number of sessions never exceeds $n - t + 1$. □

CHAPTER 5. COMBINATORIAL AND PROBABILISTIC ANALYSIS OF THE QUORUM SELECTION PROBLEM

Here we formalize quorum selection as a problem on hypergraphs, obtain combinatorial bounds (complementing the robustness proof from Chapter 5), and construct a probabilistic model based on absorbing Markov chains to estimate the expected number of sessions and the signing time.

Definition 5.1 (t-uniform complete hypergraph of quorums [27])**.** Let $V = \{1, 2, \ldots, n\}$ be the set of protocol participants, $t \leq n$ the signing threshold. The *quorum hypergraph* is the t-uniform complete hypergraph $\mathcal{H} = (V, \mathcal{E})$, where the set of hyperedges

$$\mathcal{E} = \binom{V}{t} = \{S \subseteq V : |S| = t\}$$

consists of all t-element subsets of participants. Each hyperedge $e \in \mathcal{E}$ corresponds to a potential quorum for producing a threshold signature. The total number of hyperedges equals $|\mathcal{E}| = \binom{n}{t}$.

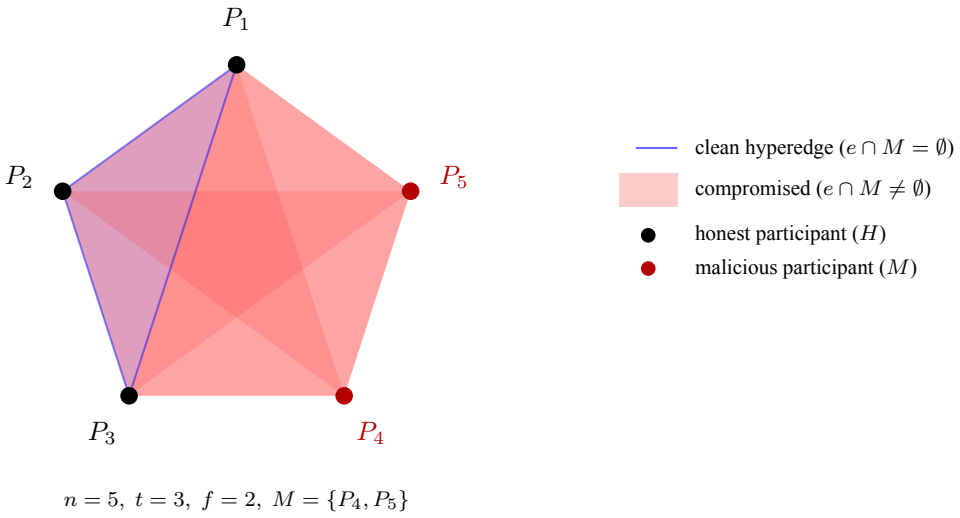

Figure 8: 3-uniform complete quorum hypergraph for $n = 5$ participants. The unique clean hyperedge $\{P_1, P_2, P_3\}$ is highlighted in blue; the remaining $\binom{5}{3} - \binom{3}{3} = 9$ hyperedges are compromised (red background).

Definition 5.2 (Clean and compromised hyperedge)**.** Let $M \subseteq V$, $|M| = f \leq n - t$, be the set of malicious participants, $H = V \setminus M$ the set of honest participants, $|H| = n - f \geq t$. A hyperedge $e \in \mathcal{E}$ is called:

- *clean*, if $e \cap M = \emptyset$ (all quorum members are honest);
- *compromised*, if $e \cap M \neq \emptyset$ (the quorum contains at least one malicious participant).

The coordinator's task in the ROAST protocol is to *adaptively search for a clean hyperedge*: the coordinator does not know M a priori, but can obtain information about individual malicious participants via the identifiable-abort property [3]. Each failed session with a compromised quorum allows at least one malicious participant to be identified and excluded from subsequent selection.

Proposition 5.1 (Number of clean quorums)**.** The number of clean hyperedges equals

$$|\mathcal{E}_{\text{clean}}| = \binom{n-f}{t},$$

and the number of compromised hyperedges is

$$|\mathcal{E}_{\text{cont}}| = \binom{n}{t} - \binom{n-f}{t}.$$

Given $f \leq n - t$, we have $|\mathcal{E}_{\text{clean}}| \geq 1$, i.e., at least one clean quorum exists.

Proof. Clean hyperedges are t-element subsets of the set of honest participants H, $|H| = n - f \geq t$. Their number equals $\binom{n-f}{t}$. Compromised ones are the rest: $\binom{n}{t} - \binom{n-f}{t}$. Since $n - f \geq t$, we have $\binom{n-f}{t} \geq \binom{t}{t} = 1$. □

Definition 5.3 (Hypergraph transversal)**.** A set $T \subseteq V$ is called a *transversal* of a hypergraph $\mathcal{H} = (V, \mathcal{E})$ if $T \cap e \neq \emptyset$ for every $e \in \mathcal{E}$.

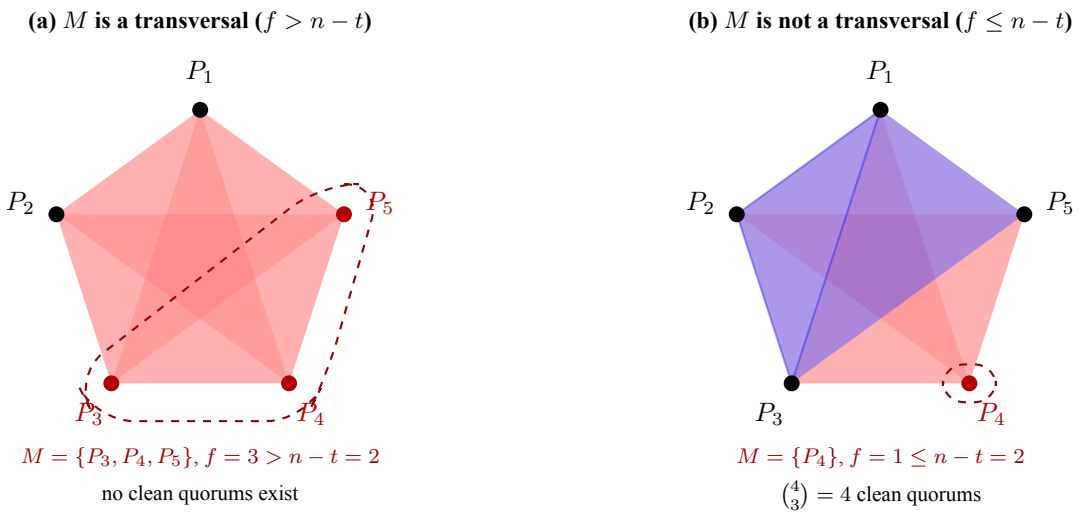

Figure 9: Transversals and the existence of clean quorums. (a) When $f > n - t$, the set M is a transversal of the subhypergraph $\mathcal{H}_{\text{clean}}$: every hyperedge contains a malicious participant. (b) When $f \leq n-t$, the set M is not a transversal: clean hyperedges exist (blue) that do not intersect M.

In our problem [28], M is a transversal of $\mathcal{H}_{\text{clean}} = (H, \binom{H}{t})$ if and only if $|H| < t$, i.e., no clean quorums exist. When $f \leq n - t$, we have $|H| \geq t$, so M is *not* a transversal and clean quorums exist. ROAST exploits this: instead of searching for a minimum transversal (NP-complete for general hypergraphs), it adaptively excludes elements of M via identifiable abort, shrinking the hypergraph at each step.

In Chapter 5 it was shown that ROAST initiates $\leq n - t + 1$ sessions. Next we prove that this bound is tight.

Theorem 5.1 (Lower bound on the number of sessions)**.** For any deterministic threshold signing protocol that sequentially selects t-element quorums and obtains information about at most one malicious participant [3] from each compromised quorum, in the worst case at least $f + 1 = n - t + 1$ sessions are necessary to guarantee the production of a signature.

Proof. We construct an adaptive adversary strategy. Let $|M| = f$, and suppose the coordinator selects quorums sequentially $e_1, e_2, \ldots$

After $k - 1$ failed sessions, the coordinator has identified $k - 1$ malicious participants; $f - (k - 1)$ malicious participants remain unknown. The adversary controls the message delivery schedule [3] and, at step k, ensures that among the t participants who respond first, exactly one is malicious: it suffices for one of the $f - (k - 1)$ unknown malicious participants to respond among the first t, while the remaining malicious participants delay their responses.

This strategy is feasible for $k = 1, \ldots, f$, since $f - (k - 1) \geq 1$. Each of the first f sessions is compromised and reveals exactly one malicious participant. At session $f + 1$, all f malicious participants have been identified and excluded, so the quorum consists exclusively of honest participants ($|H| = n - f \geq t$) and is clean.

Hence, no protocol can guarantee completion in fewer than $f + 1 = n - t + 1$ sessions. □

Corollary 5.1 (Optimality of ROAST)**.** The upper bound of $n - t + 1$ sessions from Chapter 5 coincides with the lower bound of Theorem 5.1. Hence ROAST is optimal in the number of sessions in the worst case.

Let us compare ROAST with a naive strategy that selects quorums uniformly at random and does not exclude identified malicious participants.

Proposition 5.2 (Probability of a clean quorum under random selection)**.** If a quorum of size t is chosen uniformly at

random from all $\binom{n}{t}$ possible subsets, the probability that it is clean equals

$$p_0 = \frac{\binom{n-f}{t}}{\binom{n}{t}} = \prod_{i=0}^{t-1} \frac{n-f-i}{n-i}.$$

Proof. This follows directly from Proposition 5.1: $p_0 = |\mathcal{E}_{\text{clean}}|/|\mathcal{E}|$. The product representation is obtained by expanding the binomial coefficients:

$$\frac{\binom{n-f}{t}}{\binom{n}{t}} = \frac{(n-f)!\,/\,(n-f-t)!}{n!\,/\,(n-t)!} = \prod_{i=0}^{t-1} \frac{n-f-i}{n-i}.$$

The probability p_0 corresponds to the hypergeometric distribution [29]. □

For the naive strategy without exclusion (quorums chosen independently), the expected number of attempts until the first clean quorum follows a geometric distribution with mean $1/p_0$. Indeed, each attempt is independent of the previous ones and succeeds with probability p_0; the number of attempts until the first success $X \sim \text{Geom}(p_0)$, so $\mathbb{E}[X] = 1/p_0$.

Let us give numerical values for typical configurations:

n	t	f	p_0	$1/p_0$ (naive)	$n-t+1$ (ROAST)
5	3	2	$1/10 = 0.100$	10	3
7	4	3	$1/35 \approx 0.029$	35	4
10	7	3	$1/120 \approx 0.008$	120	4
15	10	5	$1/3003 \approx 0.0003$	3003	6

The table shows that as n and f grow, the probability p_0 decreases rapidly, and the expected number of attempts under the naive strategy grows combinatorially. The adaptive ROAST approach provides a linear dependence $f+1$ instead of $1/p_0 = \binom{n}{t}/\binom{n-f}{t}$, which can be exponentially better. For example, with $n = 15$, $t = 10$, $f = 5$, the naive strategy requires on average 3003 attempts, whereas ROAST guarantees no more than 6.

We now construct a probabilistic model of the ROAST protocol using absorbing Markov chains [30] to analyze the expected number of sessions and the expected signing completion time. This model complements the deterministic worst-case analysis with average-case estimates under a random order of participant responses.

Definition 5.4 (Stochastic model of ROAST)**.** Consider the ROAST protocol with n participants, threshold t, and $f = n-t$ malicious participants. Suppose participants respond to the coordinator in a random order (uniformly random permutations); the first t to respond form the session's quorum. If the quorum is clean, a signature is produced. If the quorum is compromised, the coordinator identifies one malicious participant via ShareVal [3] and excludes them from subsequent sessions.

The model applies when each participant's response time is an independent, identically distributed random variable (e.g., network delays), and the adversary does not control the delivery schedule. The worst case (adaptive adversary) was discussed in Chapter 5 and at the beginning of Chapter 6.

Note: the model assumes exactly one malicious participant is identified per failed session. In reality, ShareVal checks each response individually, so a single compromised session may reveal several malicious participants at once. Hence E_0 from this model is an upper estimate.

Definition 5.5 (Markov chain of the number of sessions)**.** Define a discrete Markov chain $\{X_k\}_{k\geq 0}$ on the state space $\mathcal{S} = \{0, 1, \ldots, f\} \cup \{\Delta\}$, where:

- state $m \in \{0, 1, \ldots, f\}$ — the state in which the coordinator has identified and excluded exactly m malicious participants out of the total $f = n - t$. In this state, $n - m$ participants remain available, among which $n - f$ are honest and $f - m$ are not-yet-identified malicious participants;

- state Δ — the state in which a signature has been successfully produced and the protocol has terminated. Once the chain reaches Δ, it remains there forever.

Initial state: $X_0 = 0$ (no malicious participant has yet been identified). At each step k, the coordinator initiates one signing session, selecting a quorum of size t from the $n - m$ available participants. The outcome of the session determines the transition:

- with probability p_m, the quorum is *clean* (all t participants are honest) — the signature is successfully produced, and the chain transitions to the absorbing state Δ;
- with probability $1 - p_m$, the quorum is *compromised* (contains at least one malicious participant) — via identifiable abort, the coordinator identifies one malicious participant, excludes them, and the chain transitions to state $m + 1$.

Formally, the transition probabilities are:

$$P(X_{k+1} = \Delta \mid X_k = m) = p_m, \qquad P(X_{k+1} = m + 1 \mid X_k = m) = 1 - p_m,$$

where p_m is the probability that a random quorum of size t out of the $n - m$ available participants is clean. Since a clean quorum is a t-element subset of the $n - f$ honest participants, chosen from among all $\binom{n-m}{t}$ possible subsets:

$$p_m = \frac{\binom{n-f}{t}}{\binom{n-m}{t}}, \qquad m = 0, 1, \ldots, f.$$

Note that $p_f = \binom{n-f}{t} / \binom{n-f}{t} = 1$: once all f malicious participants have been identified, the next session is guaranteed to succeed. The sequence $p_0 \leq p_1 \leq \cdots \leq p_f = 1$ is increasing, since excluding malicious participants increases the fraction of honest participants among those available.

Theorem 5.2 (Expected number of sessions)**.** Let E_m be the expected number of sessions until successful completion, starting from state m. Then:

$$E_m = 1 + (1 - p_m) \cdot E_{m+1}, \qquad m = 0, 1, \ldots, f - 1, \qquad E_f = 1.$$

Unrolling the recurrence yields the closed-form expression:

$$E_0 = \sum_{m=0}^{f} \prod_{j=0}^{m-1} (1 - p_j),$$

where the product for $m = 0$ equals 1 (empty product).

Proof. From state m, the coordinator runs one session: with probability p_m the session succeeds (1 session spent); with probability $1 - p_m$ the session fails, one malicious participant is identified, and the process moves to state $m + 1$ (1 session spent, plus E_{m+1} remaining sessions). Hence:

$$E_m = p_m \cdot 1 + (1 - p_m)(1 + E_{m+1}) = 1 + (1 - p_m)E_{m+1}.$$

For the closed form, we unroll the recurrence:

$$\begin{aligned} E_0 &= 1 + (1-p_0)E_1 \\ &= 1 + (1-p_0)\big[1 + (1-p_1)E_2\big] \\ &= 1 + (1-p_0) + (1-p_0)(1-p_1) + \cdots + \prod_{j=0}^{f-1}(1-p_j) \cdot \underbrace{E_f}_{=1} \\ &= \sum_{m=0}^{f} \prod_{j=0}^{m-1}(1-p_j). \quad \square \end{aligned}$$

Proposition 5.3 (Comparison with the naive strategy)**.** The expected number of attempts under the naive strategy (selection without exclusion) equals $E_{\text{naive}} = 1/p_0$. The ratio

$$\frac{E_{\text{naive}}}{E_0} = \frac{1/p_0}{\sum_{m=0}^{f}\prod_{j=0}^{m-1}(1-p_j)}$$

characterizes the gain from adaptive exclusion in ROAST.

Let us compute E_0 for specific configurations:

Example 1. $n = 5$, $t = 3$, $f = 2$:

$$p_0 = \frac{1}{10}, \quad p_1 = \frac{1}{4}, \quad E_2 = 1, \quad E_1 = 1 + \frac{3}{4} = \frac{7}{4}, \quad E_0 = 1 + \frac{9}{10}\cdot\frac{7}{4} = \frac{103}{40} \approx 2.58.$$

Naive strategy: $1/p_0 = 10$. Gain from adaptive exclusion: $\approx 3.9\times$.

Example 2. $n = 7$, $t = 4$, $f = 3$:

$$p_0 = \frac{1}{35}, \quad p_1 = \frac{1}{15}, \quad p_2 = \frac{1}{5}, \quad E_3 = 1, \quad E_2 = \frac{9}{5}, \quad E_1 = \frac{67}{25}, \quad E_0 \approx 3.60.$$

Naive strategy: 35 attempts. Gain: $\approx 9.7\times$.

Example 3. $n = 10$, $t = 7$, $f = 3$:

$$p_0 = \frac{1}{120}, \quad p_1 = \frac{1}{36}, \quad p_2 = \frac{1}{8}, \quad E_0 \approx 3.80.$$

Naive strategy: 120 attempts. Gain: $\approx 31.6\times$.

Let us summarize the results in a table:

n	t	f	E_0 (ROAST)	$1/p_0$ (naive)	$f+1$ (worst case)	gain
5	3	2	2.58	10	3	$3.9\times$
7	4	3	3.60	35	4	$9.7\times$
10	7	3	3.80	120	4	$31.6\times$
15	10	5	5.86	3003	6	$512\times$

E_0 under a random order of responses is close to the worst-case bound $f + 1$, but much smaller than $1/p_0$ of the naive strategy. The gain increases with the parameters.

Expected signing completion time.

To estimate the expected completion time, we assume each participant's response time is an independent random variable with the exponential distribution $\text{Exp}(\lambda)$.

Proposition 5.4 (Time of a single session)**.** In state m, the coordinator waits for responses from t out of $n-m$ available participants. The time until the t-th response is received is the t-th order statistic [31] of $n-m$ independent Exp(λ) variables. Its expectation is:

$$\mathbb{E}[T_m] = \frac{1}{\lambda} \sum_{k=0}^{t-1} \frac{1}{n-m-k}.$$

Proof. Let $N = n - m$ be the number of available participants in state m, and let $Y_1, \ldots, Y_N \sim \text{Exp}(\lambda)$ be independent random variables modeling each participant's response time. Denote the order statistics $Y_{(1)} \leq Y_{(2)} \leq \cdots \leq Y_{(N)}$. The coordinator waits for the t-th response, i.e., we are interested in $\mathbb{E}[Y_{(t)}]$.

Consider the inter-arrival difference $D_k = Y_{(k)} - Y_{(k-1)}$ (where $Y_{(0)} = 0$) — the waiting time between the $(k-1)$-th and k-th responses. At the moment when $k-1$ responses have already arrived, $N-k+1$ participants remain, each of whom has not yet responded. The exponential distribution has the *memoryless* property: $P(Y_i > t+s \mid Y_i > t) = P(Y_i > s)$ for any $t, s > 0$. This means that the conditional distribution of the remaining time $Y_i - t$ given $Y_i > t$ is again Exp(λ), regardless of how much time has already elapsed. Hence, at the moment the $(k-1)$-th response arrives, each of the $N-k+1$ remaining participants has a remaining time distributed as Exp(λ), independently of one another. The time until the next (fastest) response is the minimum of $N-k+1$ independent Exp(λ) variables:

$$D_k = Y_{(k)} - Y_{(k-1)} \sim \text{Exp}\big((N-k+1)\lambda\big),$$

since the minimum of r independent Exp(λ) variables has distribution Exp($r\lambda$) [31]. Moreover, the differences $D_1, D_2, \ldots, D_N$ are mutually independent (Rényi representation).

Since $Y_{(t)} = D_1 + D_2 + \cdots + D_t$, by linearity of expectation:

$$\mathbb{E}[Y_{(t)}] = \sum_{k=1}^{t} \mathbb{E}[D_k] = \sum_{k=1}^{t} \frac{1}{(N-k+1)\lambda} = \frac{1}{\lambda} \sum_{k=0}^{t-1} \frac{1}{n-m-k}. \quad \square$$

Theorem 5.3 (Total expected time)**.** The total expected completion time of the ROAST protocol is:

$$\mathbb{E}[T_{\text{total}}] = \sum_{m=0}^{f} \left(\prod_{j=0}^{m-1} (1-p_j) \right) \cdot \mathbb{E}[T_m],$$

where the factor $\prod_{j=0}^{m-1}(1-p_j)$ is the probability that the process reaches state m (the first m sessions were compromised).

Proof. State m is reached if and only if the first m sessions (in states $0, 1, \ldots, m-1$) failed. Each session in state j fails with probability $1-p_j$. A session in state m (regardless of its outcome) takes time T_m. Summing over all possible states:

$$\mathbb{E}[T_{\text{total}}] = \sum_{m=0}^{f} P(\text{state } m \text{ reached}) \cdot \mathbb{E}[T_m] = \sum_{m=0}^{f} \left(\prod_{j=0}^{m-1} (1-p_j) \right) \cdot \mathbb{E}[T_m]. \quad \square$$

Example. For $n = 5$, $t = 3$, $f = 2$, $\lambda = 1$:

$$\begin{aligned}
\mathbb{E}[T_0] &= \frac{1}{5} + \frac{1}{4} + \frac{1}{3} = \frac{47}{60} \approx 0.783, \\
\mathbb{E}[T_1] &= \frac{1}{4} + \frac{1}{3} + \frac{1}{2} = \frac{13}{12} \approx 1.083, \\
\mathbb{E}[T_2] &= \frac{1}{3} + \frac{1}{2} + 1 = \frac{11}{6} \approx 1.833.
\end{aligned}$$

$$\mathbb{E}[T_{\text{total}}] = 1 \cdot \frac{47}{60} + \frac{9}{10} \cdot \frac{13}{12} + \frac{9}{10} \cdot \frac{3}{4} \cdot \frac{11}{6} = \frac{47}{60} + \frac{117}{120} + \frac{297}{240} \approx 0.783 + 0.975 + 1.238 = 2.996/\lambda.$$

For comparison, the time of a single guaranteed-clean session (when M is known) equals $\mathbb{E}[T_0] \approx 0.783/\lambda$. Thus, ROAST's adaptive exclusion increases the expected time by only ≈ 3.8 times relative to the ideal case in the presence of $f = 2$ malicious participants out of $n = 5$ total.

Probabilistic modeling of participant strategies.

In the previous subsections we assumed that each participant's response time is an independent, identically distributed random variable with distribution $\text{Exp}(\lambda)$. In real systems, participants have different network connection characteristics and computational resources, and malicious participants may strategically control the speed of their responses. In this subsection we generalize the model to the heterogeneous case.

Definition 5.6 (Classification of participants by behavior)**.** Each participant P_i belongs to one of three classes:

- **Honest fast** ($\mathcal{H}_f$): responds with intensity λ_f and always sends a valid partial response. Count: n_f.
- **Honest slow** ($\mathcal{H}_s$): responds with intensity $\lambda_s < \lambda_f$ (network delays, weak computational resources) and always sends a valid partial response. Count: n_s.
- **Malicious** ($\mathcal{M}$): responds with intensity μ and sends an invalid partial response (detected via ShareVal). Count: f.

Here $n_f + n_s + f = n$ and $n_f + n_s = n - f \geq t$.

Now the probability of quorum compromise depends not only on the number of malicious participants, but also on the *relative speed* of their responses. If $\mu \gg \lambda_f$, a malicious participant enters the quorum significantly more often than the homogeneous model predicts.

Theorem 5.4 (Probability of a clean quorum with heterogeneous intensities)**.** Suppose that after m malicious participants have been identified, there remains a pool of n_f fast honest (intensity λ_f), n_s slow honest (intensity λ_s), and $f_m = f - m$ not-yet-identified malicious (intensity μ) participants. Each participant's response time is an independent exponential random variable with the corresponding intensity. Then the probability that the quorum formed by the first t respondents is clean is:

$$\tilde{p}_m = \prod_{k=0}^{t-1} \frac{\Lambda_H(k)}{\Lambda_H(k) + f_m\mu},$$

where $\Lambda_H(k)$ is the total intensity of honest participants after removing k of them who have already joined the quorum:

$$\Lambda_H(k) = (n_f - k_f)\lambda_f + (n_s - k_s)\lambda_s,$$

where (k_f, k_s) is a random pair satisfying $k_f + k_s = k$. Computing $\tilde{p}_m$ exactly requires averaging over all possible sequences of honest-participant selection.

Proof. We use the property of competing exponential random variables: if $Z_1 \sim \text{Exp}(\lambda_1), \ldots, Z_r \sim \text{Exp}(\lambda_r)$ are independent, then

$$P(Z_i = \min\{Z_1, \ldots, Z_r\}) = \frac{\lambda_i}{\lambda_1 + \cdots + \lambda_r}.$$

In other words, the probability of each variable "winning" is proportional to its intensity.

We construct $\tilde{p}_m$ sequentially. Initially the pool contains $n_f + n_s$ honest participants with total intensity $\Lambda_H(0) = n_f\lambda_f + n_s\lambda_s$ and f_m malicious participants with total intensity $f_m\mu$. By the property above, the probability that the *first* respondent is honest is:

$$P(\text{1st honest}) = \frac{\Lambda_H(0)}{\Lambda_H(0) + f_m\mu}.$$

Conditional on the first respondent being honest, we remove them from the pool. The total honest intensity decreases to $\Lambda_H(1)$ (the specific value depends on whether a fast or slow participant dropped out). The number of malicious participants does not change, since we condition on none of them having responded yet. The probability that the *second* respondent is also honest (given the first was honest) is:

$$P(\text{2nd honest} \mid \text{1st honest}) = \frac{\Lambda_H(1)}{\Lambda_H(1) + f_m\mu}.$$

Continuing inductively for steps $k = 0, 1, \ldots, t-1$, we obtain the probability that *all* t first respondents are honest as the product of conditional probabilities:

$$\tilde{p}_m = \prod_{k=0}^{t-1} P((k{+}1)\text{-th honest} \mid \text{first } k \text{ honest}) = \prod_{k=0}^{t-1} \frac{\Lambda_H(k)}{\Lambda_H(k) + f_m\mu}.$$

Note that $f_m\mu$ does not change at any step, since the appearance of a malicious participant among the respondents immediately makes the quorum compromised — we are computing exactly the probability of avoiding this event at each of the t steps. Conditional on the k-th honest respondent being fast (with probability $n_f\lambda_f/\Lambda_H(0)$) or slow (with probability $n_s\lambda_s/\Lambda_H(0)$), the pair (k_f, k_s) determines the specific value of $\Lambda_H(k)$. Computing $\tilde{p}_m$ exactly requires averaging over all possible selection sequences. □

To simplify the analytical treatment, consider a two-group model in which all honest participants share the same intensity λ, and malicious participants have intensity μ. Define the *malicious speed advantage* parameter:

$$\alpha = \frac{\mu}{\lambda}.$$

Corollary 5.2 (Homogeneous model with speed advantage)**.** With identical intensity λ for honest participants and $\mu = \alpha\lambda$ for malicious ones:

$$p_m(\alpha) = \prod_{k=0}^{t-1} \frac{(n-f)-k}{(n-f)-k+(f-m)\cdot\alpha}.$$

At $\alpha = 1$ this coincides with p_m of the homogeneous model. At $\alpha > 1$ (fast malicious participant) the probability of a clean quorum decreases; at $\alpha < 1$ (slow malicious participant) it increases.

Proof. With homogeneous intensities, $\Lambda_H(k) = ((n-f)-k)\lambda$, hence:

$$p_m(\alpha) = \prod_{k=0}^{t-1} \frac{((n-f)-k)\lambda}{((n-f)-k)\lambda + (f-m)\alpha\lambda} = \prod_{k=0}^{t-1} \frac{(n-f)-k}{(n-f)-k+(f-m)\alpha}.$$

At $\alpha = 1$:

$$p_m(1) = \prod_{k=0}^{t-1} \frac{(n-f)-k}{(n-m)-k} = \frac{(n-f)!/(n-f-t)!}{(n-m)!/(n-m-t)!} = \frac{\binom{n-f}{t}}{\binom{n-m}{t}}. \quad \square$$

Let us give numerical values of $E_0(\alpha)$ for the configuration $n = 7$, $t = 4$, $f = 3$:

α	$p_0(\alpha)$	$E_0(\alpha)$	$f+1$	interpretation
0.5	0.1108	3.02	4	slow malicious participant
1.0	0.0286	3.60	4	homogeneous model
2.0	0.0048	3.89	4	fast malicious participant
5.0	0.0003	3.99	4	very fast malicious participant
∞	0	4.00	4	worst case

The table shows that even as $\alpha \to \infty$ (the malicious participant responds instantly and is guaranteed to enter the

quorum), the expected number of sessions converges to $f + 1 = 4$, which is the deterministic upper bound. Adaptive exclusion via identifiable abort ensures *boundedness* $E_0(\alpha) \leq f + 1$ for any α, whereas the naive strategy (without exclusion) requires $1/p_0(\alpha) \to \infty$ as $\alpha \to \infty$.

Effect of heterogeneous honest participants. In the full model with n_f fast (λ_f) and n_s slow (λ_s) honest participants, the probability of quorum compromise increases if slow honest participants "yield their place" to fast malicious participants. Formally, for a fixed total honest intensity $n_f\lambda_f + n_s\lambda_s = \Lambda$, the probability of contamination is maximized when the variance of intensities is maximized. This motivates a practical recommendation: in ROAST deployments, it is desirable to minimize the variability of honest participants' response times by optimizing the network infrastructure.

The basic ROAST algorithm (Chapter 5) does not use historical information about participant behavior: the coordinator forms a quorum from the first t respondents, without prioritization. An extension is the introduction of a *reputation system*, in which the coordinator gives priority to participants with a successful participation history.

Definition 5.7 (Reputation system for ROAST)**.** Let $w_i^{(k)} \in [0, W_{\text{max}}]$ be the reputation weight of participant P_i after the k-th session. The coordinator updates the weights according to the rule:

- If P_i provided a valid partial response in session k, then $w_i^{(k+1)} = \min\big(w_i^{(k)} + \delta_+,\ W_{\text{max}}\big)$;
- If ShareVal detected an invalid response from P_i, then $w_i^{(k+1)} = 0$ and P_i is excluded from subsequent sessions (as in the base ROAST);
- For participants who did not take part in session k, $w_i^{(k+1)} = \gamma \cdot w_i^{(k)}$, where $\gamma \in (0, 1)$ is the forgetting coefficient.

Initial weights: $w_i^{(0)} = w_0 > 0$ for all i.

Modification of quorum selection. Instead of uniformly random selection (or selecting the first t respondents), the coordinator forms a quorum by *weighted sampling without replacement*: the probability of including P_i in the quorum is proportional to w_i. This is equivalent to a model in which the effective response intensity of P_i is multiplied by w_i:

$$\tilde{\lambda}_i = w_i \cdot \lambda_i.$$

Proposition 5.5 (Effectiveness of reputation for honest participants)**.** Suppose that after K successful sessions an honest participant P_i has reputation $w_i^{(K)} = \min(w_0 + K\delta_+, W_{\text{max}})$, while a new (or inactive) participant has reputation w_0 or less. Then the effective intensity of a proven honest participant exceeds that of a new, unknown participant by a factor of $\min(w_0 + K\delta_+, W_{\text{max}})/w_0$, which increases the probability of forming a clean quorum from proven participants.

Risk of a "sleeper agent". A malicious participant may behave honestly for K sessions, accumulate reputation $w_{\text{mal}} = \min(w_0 + K\delta_+, W_{\text{max}})$, and then block a session with an invalid response at the desired moment.

Definition 5.8 (Sleeper agent strategy)**.** A malicious participant $P_j \in \mathcal{M}$ executes the strategy $\mathcal{S}_K$: during the first K sessions it behaves honestly (sends valid partial responses), and at session $K + 1$ it sends an invalid response.

Proposition 5.6 (Bounded harm of a sleeper agent)**.** The strategy $\mathcal{S}_K$ guarantees the malicious participant's presence in the quorum of session $K + 1$ with high probability (thanks to accumulated reputation), but its harm is limited to a single failed session: once ShareVal detects the invalid response, participant P_j is excluded permanently ($w_j = 0$, added to the set M). Therefore:

1. each malicious participant can successfully execute the strategy $\mathcal{S}_K$ at most once;
2. the total number of sessions blocked by sleeper agents does not exceed f;
3. the upper bound of $f + 1$ sessions for successful signing is preserved.

Proof. After sending an invalid response, P_j is identified via ShareVal and added to M, regardless of its previous reputation. Since $|\mathcal{M}| = f$, at most f sessions can be blocked by sleeper agents. At session $f+1$, all malicious participants have already been excluded (either identified earlier or having executed $\mathcal{S}_{K_j}$), so the quorum consists exclusively of honest participants. □

Remark 5.1 (Optimal forgetting strategy)**.** The forgetting coefficient $\gamma \in (0, 1)$ balances two extremes:

- as $\gamma \to 1$ (no forgetting): reputation accumulates without bound, which maximizes efficiency for stable honest participants but increases the impact of sleeper agents ($w_{\text{mal}} \approx W_{\max}$ after sufficient K);
- as $\gamma \to 0$ (strong forgetting): reputation is quickly "forgotten", which minimizes the impact of sleeper agents but reduces the advantage of proven participants over new ones.

The effective memory horizon of the system equals $\tau = 1/(1-\gamma)$ sessions. To protect against the strategy $\mathcal{S}_K$, it is recommended that $W_{\max} \le C \cdot w_0$ for a small constant $C > 1$ (for example, $C = 3$), which bounds the maximum advantage a malicious participant can gain by accumulating reputation.

Justification. The reputation of a participant who has not taken part in the last ℓ sessions decays as $w_i^{(k+\ell)} = \gamma^\ell w_i^{(k)}$. After $\tau = 1/(1-\gamma)$ inactive sessions, the reputation decreases by a factor of e ($\gamma^\tau \approx e^{-1}$). The bound $W_{\max} \le C \cdot w_0$ guarantees that even a participant with maximal reputation has an effective intensity at most C times that of a new participant, which bounds the probability of a sleeper agent being included in the quorum by the factor $C/(C+(n-f-1)\cdot 1)$ instead of $1/(n-m)$ in the homogeneous model.

Let us summarize the comparative analysis of coordinator strategies:

strategy	E_0	protection against sleeper agent
base ROAST	E_0^{base} (Thm. 5.2)	$f+1$ sessions
reputation ($\gamma = 1$)	$\le E_0^{\text{base}}$	$f+1$ sessions
reputation ($\gamma < 1$)	$\le E_0^{\text{base}}$	$f+1$ sessions

The table shows that the reputation system *does not change* the worst-case upper bound of $f+1$ sessions (since even a sleeper agent is eventually detected and excluded), but it can improve the *average* number of sessions E_0, since high-reputation honest participants form a quorum faster. The key invariant is that the ShareVal mechanism of the ROAST protocol guarantees the inevitability of excluding a malicious participant, regardless of the reputation strategy.

SOFTWARE IMPLEMENTATION

All three protocols — FROST, ChillDKG, and ROAST — have been implemented in Go on the Edwards25519 curve; Ristretto255 group arithmetic is provided by the `filippo.io/edwards25519` library. The source code is available in the repository [13].

The implementation includes EdDSA primitives (shares, keys, signature verification), polynomial arithmetic in $\mathbb{Z}_q[x]$, and Schnorr proofs of key possession. The FROST signing protocol runs in two rounds: participants generate nonce commitments, then compute partial signatures, which the coordinator aggregates into an Ed25519-compatible signature. ChillDKG consists of three layers — SimplPedPop, EncPedPop, CertEq — and allows a share to be recovered from a stored seed. The ROAST coordinator runs parallel sessions, excludes malicious participants via ShareVal, and ranks participants by reputation with a forgetting coefficient γ.

The correctness of the theoretical results was verified by computational experiments. In particular, it was confirmed that $t-1$ participants cannot recover the secret after ChillDKG, that the protocol correctly identifies a violator and generates a verifiable proof, and that CertEq terminates either in consensus or in an identified abort. Integration tests run the full cycle (key generation, signing, Ed25519 verification) for several (t, n) configurations, including failure scenarios.

CONCLUSIONS

This thesis addresses the problem of robustness of asynchronous threshold Schnorr signatures. Below we summarize the results obtained, in line with the objectives formulated in the introduction.

Chapter 3 examines the FROST protocol for (t, n)-threshold signing in detail. We analyzed the commitment-binding mechanism, which protects against forgery attacks, and the construction based on two polynomials $F_1(x)$ and $F_2(x)$, which guarantees the correctness of the group signature via Lagrange interpolation.

In Chapter 4, the classical Pedersen DKG is replaced by ChillDKG, which removes the dependence on an external broadcast channel and a consensus mechanism. For this protocol, the thesis proves statements regarding: resistance to threshold escalation, correctness and completeness of malicious-participant identification, and public verifiability of proofs of malicious behavior. Separately, we prove the impossibility of identifiable abort in 1 round and show that 3 logical rounds of ChillDKG are both necessary and sufficient.

Chapter 5 describes the ROAST extension, which turns FROST into a robust asynchronous protocol. In Chapter 6, the quorum selection problem is formalized in the language of t-uniform hypergraphs, where hyperedges correspond to quorums and the coordinator adaptively searches for a clean hyperedge, excluding malicious participants. For the upper bound of $n - t + 1$ sessions, optimality is proven: an adaptive adversary strategy is constructed that forces exactly f failed sessions before the first successful one.

There, we also construct a probabilistic model based on absorbing Markov chains, with the closed-form expression $E_0 = \sum_{m=0}^{f} \prod_{j=0}^{m-1}(1-p_j)$ for the expected number of sessions, and a formula for the expected completion time via order statistics. For the configuration $n = 15, t = 10, f = 5$, adaptive exclusion yields a 512-fold gain compared to naive random quorum selection.

The model is generalized to the case of heterogeneous participant response intensities. The parameter $\alpha = \mu/\lambda$ characterizes the malicious participant's speed advantage; even as $\alpha \to \infty$, the number of sessions never exceeds $f + 1$. Additionally, a reputation system with forgetting coefficient γ is proposed, and it is shown that the sleeper-agent strategy does not worsen the upper bound on robustness.

Beyond the theoretical results, the FROST, ChillDKG, and ROAST protocols have been implemented in Go on the Edwards25519 curve. The implementation includes integration tests and computational experiments that confirm the correctness of the proven statements.

The novelty of this work lies in the original proofs for ChillDKG (five theorems on security and round optimality) and for ROAST (formalization on hypergraphs, proof of optimality of the $n - t + 1$ bound, and a probabilistic model generalized to heterogeneous intensities and a reputation system). The material of this thesis can be used in the development of threshold cryptographic systems in blockchain infrastructure, distributed key storage systems, and in the design of fault-tolerant signing protocols.

REFERENCES

[1] Josh Benaloh and Jerry Leichter. "Generalized Secret Sharing and Monotone Functions". In: *Advances in Cryptology — CRYPTO '88*. Vol. 403. Lecture Notes in Computer Science. Springer, 1988, pp. 27–35. DOI: 10.1007/0-387-34799-2_3.

[2] Chelsea Komlo and Ian Goldberg. *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*. Tech. rep. Cryptology ePrint Archive, Report 2020/852. University of Waterloo and Zcash Foundation, Dec. 2020. URL: https://eprint.iacr.org/2020/852.

[3] Tim Ruffing et al. "ROAST: Robust Asynchronous Schnorr Threshold Signatures". In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. CCS '22. ACM, 2022, pp. 2551–2564. DOI: 10.1145/3548606.3560583.

[4] Adi Shamir. “How to Share a Secret”. In: *Communications of the ACM* 22.11 (1979), pp. 612–613. DOI: 10.1145/359168.359176.

[5] Paul Feldman. “A Practical Scheme for Non-interactive Verifiable Secret Sharing”. In: *Proceedings of the 28th Annual Symposium on Foundations of Computer Science (SFCS ’87)*. IEEE Computer Society, 1987, pp. 427–438. DOI: 10.1109/SFCS.1987.4.

[6] Torben P. Pedersen. “A Threshold Cryptosystem without a Trusted Party (Extended Abstract)”. In: *Advances in Cryptology — EUROCRYPT ’91*. Vol. 547. Lecture Notes in Computer Science. Springer, 1991, pp. 522–526. DOI: 10.1007/3-540-46416-6_47.

[7] Rosario Gennaro et al. “Secure Distributed Key Generation for Discrete-Log Based Cryptosystems”. In: *Journal of Cryptology* 20 (2007), pp. 51–83. DOI: 10.1007/s00145-006-0347-3.

[8] Claus P. Schnorr. “Efficient Identification and Signatures for Smart Cards”. In: *Advances in Cryptology — CRYPTO ’89*. Ed. by Gilles Brassard. Vol. 435. Lecture Notes in Computer Science. Springer, 1990, pp. 239–252. DOI: 10.1007/0-387-34805-0_22.

[9] Elizabeth Crites, Chelsea Komlo, and Mary Maller. *How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures*. Cryptology ePrint Archive, Report 2021/1375. 2021. URL: https://eprint.iacr.org/2021/1375.

[10] Manu Drijvers et al. “On the Security of Two-Round Multi-Signatures”. In: *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 1084–1101. DOI: 10.1109/SP.2019.00050.

[11] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process”. In: *Journal of the ACM* 32.2 (1985), pp. 374–382. DOI: 10.1145/3149.214121.

[12] Jonas Nick, Tim Ruffing, and Elliott Jin. *BIP Draft: ChillDKG – Distributed Key Generation for FROST*. https://github.com/BlockstreamResearch/bip-frost-dkg. 2024.

[13] Станіслав Каращук. *frost-ed25519: Реалізація протоколів FROST, ChillDKG та ROAST на кривій Edwards25519*. https://github.com/staaason/frost-ed25519. 2026.

[14] R. Gennaro and S. Goldfeder. “Fast Multiparty Threshold ECDSA with Fast Trustless Setup”. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 1179–1194. DOI: 10.1145/3243734.3243859.

[15] Daniel J. Bernstein et al. “High-Speed High-Security Signatures”. In: *Journal of Cryptographic Engineering* 2.2 (2012), pp. 77–89. DOI: 10.1007/s13389-012-0027-1.

[16] Mike Hamburg, Isis Lovecruft, and Henry de Valence. *The Ristretto Group*. https://ristretto.group/ristretto.html. 2019.

[17] Mike Hamburg. *Decaf: Eliminating cofactors through point compression*. Cryptology ePrint Archive, Report 2015/673. Minor revision of a CRYPTO 2015 paper. ©2015 IACR. 2015. URL: https://eprint.iacr.org/2015/673.

[18] M. D. Ryan. *Computer Security Lecture Notes*. 2008. URL: https://www.cs.bham.ac.uk/~mdr/teaching/modules/security/lectures/public_key.html.

[19] Simon Josefsson and Ilari Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032. Internet Engineering Task Force, Jan. 2017. URL: https://www.rfc-editor.org/rfc/rfc8032.

[20] Ronald Cramer, Ivan Damgård, and Yuval Ishai. “Share Conversion, Pseudorandom Secret-Sharing and Applications to Secure Computation”. In: *Theory of Cryptography*. 2005, pp. 342–362. DOI: 10.1007/978-3-540-30576-7_19.

[21] Torben P. Pedersen. “Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing”. In: *CRYPTO ’91*. 1991, pp. 129–140. DOI: 10.1007/3-540-46766-1_9.

[22] Amos Fiat and Adi Shamir. “How to Prove Yourself: Practical Solutions to Identification and Signature Problems”. In: *Advances in Cryptology — CRYPTO ’86*. Vol. 263. Lecture Notes in Computer Science. Springer, 1987, pp. 186–194. DOI: 10.1007/3-540-47721-7_12.

[23] Rosario Gennaro et al. “Secure Applications of Pedersen’s Distributed Key Generation Protocol”. In: *Topics in Cryptology — CT-RSA 2003*. 2003, pp. 373–390. DOI: 10.1007/3-540-36563-X_26.

[24] Hien Chu et al. *Practical Schnorr Threshold Signatures Without the Algebraic Group Model*. Cryptology ePrint Archive, Report 2023/899. 2023. URL: https://eprint.iacr.org/2023/899.

[25] Fredrik Dahlgren. *Breaking the shared key in threshold signature schemes*. Trail of Bits. Feb. 2024. URL: https://blog.trailofbits.com/2024/02/20/breaking-the-shared-key-in-threshold-signature-schemes/.

[26] Leslie Lamport, Robert Shostak, and Marshall Pease. “The Byzantine Generals Problem”. In: *ACM Transactions on Programming Languages and Systems* 4.3 (1982), pp. 382–401. DOI: 10.1145/357172.357176.

[27] Claude Berge. *Hypergraphs: Combinatorics of Finite Sets*. Elsevier, 1989. ISBN: 9780444874894.

[28] Moni Naor and Avishai Wool. “The Load, Capacity, and Availability of Quorum Systems”. In: *SIAM Journal on Computing* 27.2 (1998), pp. 423–447. DOI: 10.1137/S0097539795281232.

[29] Norman L. Johnson, Adrienne W. Kemp, and Samuel Kotz. *Univariate Discrete Distributions*. 3rd. Wiley, 2005. ISBN: 9780471272465. DOI: 10.1002/0471715816.

[30] John G. Kemeny and J. Laurie Snell. *Finite Markov Chains*. New York: Springer-Verlag, 1976. ISBN: 9780387901923.

[31] Herbert A. David and Haikady N. Nagaraja. *Order Statistics*. 3rd. Wiley, 2003. ISBN: 9780471389262. DOI: 10.1002/0471722162.